

MNIST Overfitting and Regularization Analysis

Introduction

For this task, I set out to investigate how and why neural networks overfit on training data, and what practical steps can be taken to mitigate this behavior. I chose the well-known MNIST handwritten digit dataset and selected a fully-connected Multi-Layer Perceptron (MLP) as the primary architecture.

A fully-connected MLP contains no convolutional layers and has none of the spatial inductive biases (such as weight sharing or translation equivariance) that naturally resist overfitting. This makes it the ideal architecture family for demonstrating and analyzing overfitting clearly and holistically on both loss and accuracy curves. Everything below uses the MLP architecture family throughout, including the baseline, so that the only variable that changes between experiments is capacity and training protocol (not the architecture family itself).

The study is organized into three experiments, all using the full 70,000-image MNIST pool with an identical train/validation/test split:

1. **Baseline:** A compact, appropriately sized MLP trained without any tricks on the full training set. This confirms that MNIST itself is not difficult to generalize on when model capacity is well matched.
2. **Part A:** A deliberately over-parameterized MLP (same input data, same split) trained without any regularization. The goal is to observe overfitting that occurs purely from architectural excess, with nothing about the data or split changed.
3. **Part B:** The identical over-parameterized architecture from Part A (with not one parameter changed) trained on the same data, but now with data augmentation, weight decay, early stopping, and a learning rate scheduler added to the training protocol. This demonstrates how training-time regularization alone can recover generalization without touching the architecture.

All three experiments are compared side-by-side through training curves, test set evaluations, confusion matrices, and per-class accuracy charts. Training progress for every model is displayed live with `tqdm` progress bars showing per-batch loss and running accuracy.

1. Importing Libraries and Device Selection

```
import torch
import torch.nn as nn
import torch.optim as optim
```

```
from torch.utils.data import DataLoader, random_split, Subset, ConcatDataset
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import matplotlib as mpl
import numpy as np
import time
import copy
import itertools
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import train_test_split
from tqdm.auto import tqdm
import seaborn as sns

# Set random seed for reproducibility
torch.manual_seed(42)
np.random.seed(42)

# Check for GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Device: {device}")
print(f"PyTorch Version: {torch.__version__}")
if device.type == 'cuda':
    print(f"GPU: {torch.cuda.get_device_name(0)}")

# Global plot styling -- applied to every figure in the notebook
sns.set_theme(style="whitegrid", palette="deep")
mpl.rcParams.update({
    "figure.facecolor": "white",
    "axes.facecolor": "white",
    "axes.edgecolor": "#333333",
    "axes.titlesize": 13,
    "axes.titleweight": "bold",
    "axes.labelsiz": 11,
    "font.size": 10.5,
    "legend.frameon": True,
    "legend.framealpha": 0.9,
```

```

    "grid.alpha": 0.35,
    "lines.linewidth": 2.0,
    "lines.markersize": 5,
})

COLOR_UNREG_TRAIN    = "#1F77B4"    # blue
COLOR_UNREG_VAL      = "#D62728"    # red
COLOR_BASELINE_TRAIN = "#9467BD"    # purple
COLOR_BASELINE_VAL   = "#9467BD"    # purple (same shade, distinguished by linestyle in the plot)
COLOR_REG_TRAIN       = "#2CA02C"   # green
COLOR_REG_VAL         = "#2CA02C"   # green (same shade, distinguished by linestyle in the plot)

Device: cuda
PyTorch Version: 2.11.0+cu128
GPU: Tesla T4

```

2. Dataset Split and Preprocessing

To construct the dataset splits, I merged all 70,000 MNIST samples (60,000 training + 10,000 test) into a single pool. From this combined pool, I performed a stratified split to divide the data into:

- 70% for training (49,000 images)
- 15% for validation (10,500 images)
- 15% for testing (10,500 images)

The stratified approach guarantees that each of the three subsets maintains an identical proportion of the 10 digit classes, preventing any class imbalance from skewing the results. All images are normalized using the standard MNIST channel mean (0.1307) and standard deviation (0.3081).

Additionally, to study overfitting on a smaller dataset, we extract a stratified 8,000-image subset from the training set. This smaller dataset pool mimics scenarios where data is scarce relative to model capacity, leading to rapid overfitting.

```

# Preprocessing transform
basic_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

# Download dataset parts and merge them
train_dataset_std = datasets.MNIST(root='./data', train=True, download=True,
transform=basic_transform)
test_dataset_std = datasets.MNIST(root='./data', train=False, download=True,
transform=basic_transform)
full_dataset_std = ConcatDataset([train_dataset_std, test_dataset_std])

# Extract labels for stratified split
all_targets = torch.cat([train_dataset_std.targets, test_dataset_std.targets]).numpy()

# Perform stratified splits (70% train, 30% val+test)
indices = np.arange(len(all_targets))
train_idx, val_test_idx = train_test_split(
    indices, test_size=0.3, stratify=all_targets, random_state=42
)

# Split val+test into validation (15% total) and testing (15% total)
val_test_targets = all_targets[val_test_idx]
val_idx, test_idx = train_test_split(
    val_test_idx, test_size=0.5, stratify=val_test_targets, random_state=42
)

train_set = Subset(full_dataset_std, train_idx)
val_set = Subset(full_dataset_std, val_idx)
test_set = Subset(full_dataset_std, test_idx)

BATCH_SIZE = 64

```

```

train_loader = DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=True, num_workers=0)
val_loader   = DataLoader(val_set,   batch_size=256,      shuffle=False, num_workers=0)
test_loader  = DataLoader(test_set,  batch_size=256,      shuffle=False, num_workers=0)

print(f"Training set:      {len(train_set)} images")
print(f"Validation set:    {len(val_set)} images")
print(f"Test set:          {len(test_set)} images")
print(f"Image shape:       {full_dataset_std[0][0].shape} (C, H, W)")

100%|██████████| 9.91M/9.91M [00:02<00:00, 4.91MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 130kB/s]
100%|██████████| 1.65M/1.65M [00:01<00:00, 1.25MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 10.5MB/s]

Training set:      49000 images
Validation set:    10500 images
Test set:          10500 images
Image shape:       torch.Size([1, 28, 28]) (C, H, W)

```

3. Visualizing the Data

```

# Plot some sample images
fig, axes = plt.subplots(2, 10, figsize=(15, 4))
for i in range(20):
    ax = axes[i // 10, i % 10]
    image, label = train_set[i]
    raw_img = (image.squeeze() * 0.3081 + 0.1307).numpy()
    ax.imshow(raw_img, cmap='gray')
    ax.set_title(str(label), fontsize=10, fontweight='bold')
    ax.axis('off')
plt.suptitle("Samples from Training Set", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# Plot the distribution of classes in train split
train_labels = all_targets[train_idx]

```

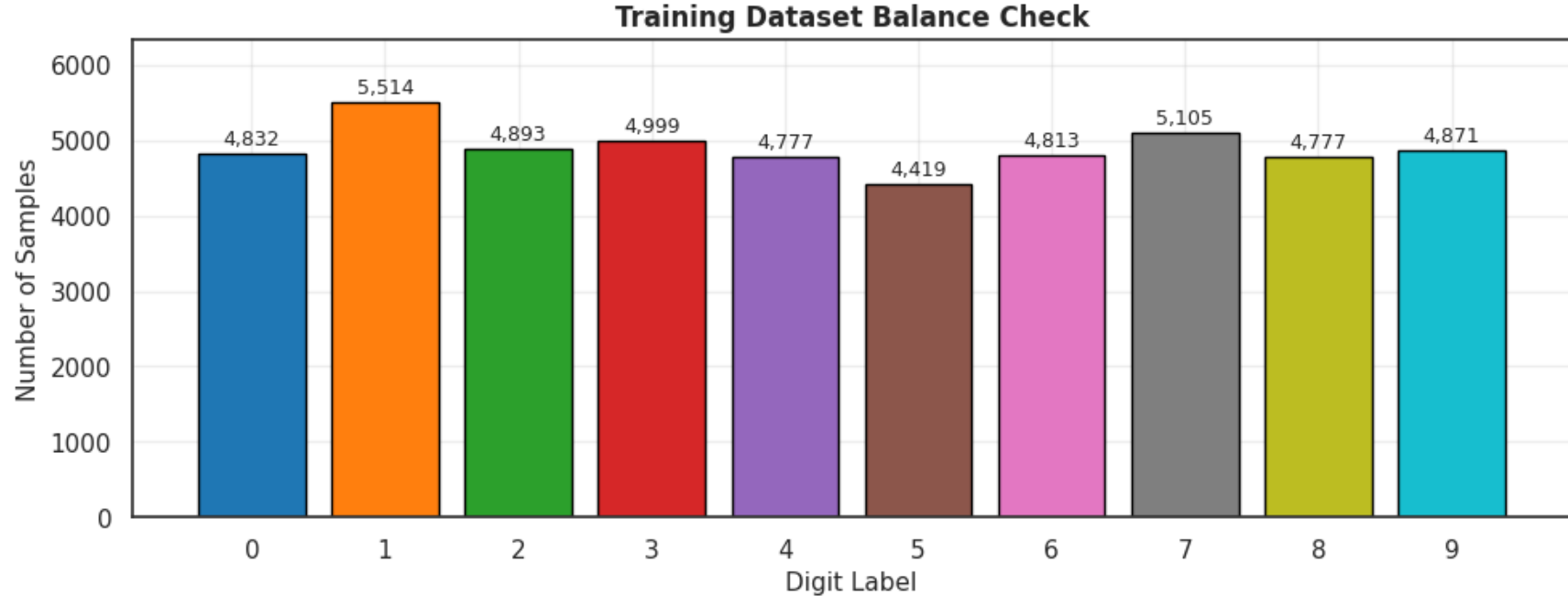
```

classes, frequency = np.unique(train_labels, return_counts=True)

fig, ax = plt.subplots(figsize=(10, 4))
bars = ax.bar(classes, frequency, color=plt.cm.tab10(np.arange(10)), edgecolor='black')
ax.set_xlabel("Digit Label")
ax.set_ylabel("Number of Samples")
ax.set_title("Training Dataset Balance Check", fontsize=12, fontweight='bold')
ax.set_xticks(range(10))
ax.set_ylim(0, max(frequency) * 1.15)
for bar, val in zip(bars, frequency):
    ax.text(bar.get_x() + bar.get_width()/2., bar.get_height() + 50,
            f"{val:,}", ha='center', va='bottom', fontsize=9)
plt.tight_layout()
plt.show()

```





4. Helper Functions for Training and Evaluation

```
def run_single_epoch(model, loader, loss_fn, opt, dev, epoch_idx, total_epochs):  
    model.train()  
    running_loss = 0.0  
    correct_preds = 0  
    total_samples = 0  
    progress_bar = tqdm(loader, desc=f"Epoch {epoch_idx+1:02d}/{total_epochs}", leave=False)  
    for inputs, targets in progress_bar:  
        inputs, targets = inputs.to(dev), targets.to(dev)  
        opt.zero_grad()  
        outputs = model(inputs)  
        loss = loss_fn(outputs, targets)  
        loss.backward()  
        opt.step()
```

```

        running_loss += loss.item() * inputs.size(0)
        _, preds = outputs.max(1)
        correct_preds += preds.eq(targets).sum().item()
        total_samples += targets.size(0)
        current_acc = 100.0 * correct_preds / total_samples
        progress_bar.set_postfix(loss=f"{loss.item():.4f}", acc=f"{current_acc:.2f}%")
    return running_loss / total_samples, 100.0 * correct_preds / total_samples

def evaluate_performance(model, loader, loss_fn, dev):
    model.eval()
    running_loss = 0.0
    correct_preds = 0
    total_samples = 0
    with torch.no_grad():
        for inputs, targets in loader:
            inputs, targets = inputs.to(dev), targets.to(dev)
            outputs = model(inputs)
            loss = loss_fn(outputs, targets)
            running_loss += loss.item() * inputs.size(0)
            _, preds = outputs.max(1)
            correct_preds += preds.eq(targets).sum().item()
            total_samples += targets.size(0)
    return running_loss / total_samples, 100.0 * correct_preds / total_samples

def train_network(model, train_loader, val_loader, loss_fn, opt, epochs, dev,
                  scheduler=None, early_stop_patience=None):
    history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': [],
              'learning_rates': []}
    best_val_loss = float('inf')
    best_weights = None
    no_improvement_epochs = 0
    start_time = time.time()
    for epoch in range(epochs):
        epoch_start = time.time()
        t_loss, t_acc = run_single_epoch(model, train_loader, loss_fn, opt, dev, epoch, epochs)

```



```

v_loss, v_acc = evaluate_performance(model, val_loader, loss_fn, dev)
current_lr = opt.param_groups[0]['lr']
history['train_loss'].append(t_loss)
history['train_acc'].append(t_acc)
history['val_loss'].append(v_loss)
history['val_acc'].append(v_acc)
history['learning_rates'].append(current_lr)
if scheduler is not None:
    if isinstance(scheduler, optim.lr_scheduler.ReduceLROnPlateau):
        scheduler.step(v_loss)
    else:
        scheduler.step()
epoch_time = time.time() - epoch_start
print(f"\nEpoch {epoch+1:02d}/{epochs} | Train Loss: {t_loss:.4f} | Train Acc:
{t_acc:.2f}% | "
      f"Val Loss: {v_loss:.4f} | Val Acc: {v_acc:.2f}% | LR: {current_lr:.6f} | Time:
{epoch_time:.1f}s")
if early_stop_patience is not None:
    if v_loss < best_val_loss:
        best_val_loss = v_loss
        best_weights = copy.deepcopy(model.state_dict())
        no_improvement_epochs = 0
    else:
        no_improvement_epochs += 1
        if no_improvement_epochs >= early_stop_patience:
            print(f"\nEarly stopping triggered at epoch {epoch+1}. "
                  f"Best val loss: {best_val_loss:.4f}")
            model.load_state_dict(best_weights)
            break
total_time = time.time() - start_time
print(f"Training finished in {total_time:.1f} seconds")
return history

def plot_metrics(history, label_prefix, train_color='#1f77b4', val_color='#d62728'):
    """Plot loss and accuracy curves for a single model's training history,

```

with the train/val gap shaded so overfitting (or the lack of it) is immediately visible."""

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
epochs = range(1, len(history['train_loss']) + 1)

ax1.plot(epochs, history['train_loss'], color=train_color, label='Training Loss')
ax1.plot(epochs, history['val_loss'], color=val_color, label='Validation Loss')
ax1.fill_between(epochs, history['train_loss'], history['val_loss'], alpha=0.12,
color=val_color)
ax1.set_xlabel('Epoch'); ax1.set_ylabel('Loss')
ax1.set_title(f"{label_prefix}: Loss History")
ax1.legend(); ax1.grid(True, linestyle='--', alpha=0.4)

ax2.plot(epochs, history['train_acc'], color=train_color, label='Training Accuracy')
ax2.plot(epochs, history['val_acc'], color=val_color, label='Validation Accuracy')
ax2.fill_between(epochs, history['train_acc'], history['val_acc'], alpha=0.12,
color=val_color)
ax2.set_xlabel('Epoch'); ax2.set_ylabel('Accuracy (%)')
ax2.set_title(f"{label_prefix}: Accuracy History")
ax2.legend(); ax2.grid(True, linestyle='--', alpha=0.4)

plt.suptitle(f"{label_prefix} -- Training Progress", fontsize=14, fontweight='bold', y=1.03)
plt.tight_layout()
plt.show()

gap = history['train_acc'][-1] - history['val_acc'][-1]
fig, ax = plt.subplots(figsize=(4.5, 4))
bars = ax.bar(['Train', 'Val'], [history['train_acc'][-1], history['val_acc'][-1]],
color=[train_color, val_color], edgecolor='black', width=0.55)
for b, v in zip(bars, [history['train_acc'][-1], history['val_acc'][-1]]):
    ax.text(b.get_x() + b.get_width()/2, v + 0.3, f"{v:.2f}%", ha='center', fontsize=10,
fontweight='bold')
ax.set_ylabel('Accuracy (%)')
ax.set_title(f"{label_prefix}\nFinal Gap: {gap:.2f}%", fontsize=11, fontweight='bold')
ax.set_ylim(min(history['val_acc'][-1], history['train_acc'][-1]) - 5, 100.5)
```

```
ax.grid(True, axis='y', linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()
```

5. Baseline Model

Before deliberately inducing overfitting, it is important to establish a **baseline** - a small, appropriately sized model. This baseline serves a critical role in the overall analysis: if a properly sized model generalizes cleanly, then any overfitting observed later in Part A can be attributed directly to overparameterization rather than to the dataset, training procedure, or architecture family.

The baseline is a small MLP: the image is flattened to 784 features and passed through a single hidden layer of 64 units, followed by BatchNorm1d (included as standard training practice to keep activations stable, not as a regularizer), ReLU, and a 20% Dropout layer. No weight decay, early stopping, or data augmentation is applied. Using an MLP here - the same architectural family as the Part A/B model in Sections 6-8 - means the only variables that change between this baseline and the deliberately-overfit model are capacity, epoch count, and dropout, not the architecture family itself.

At roughly **1/85th the parameter count** of the Part A/B model (51,018 vs. 4,364,554), this architecture is well matched to 28x28 grayscale digit classification. The baseline model is trained on the full 49,000-image training set for 12 epochs.

```
class BaselineMLP(nn.Module):
    def __init__(self):
        super(BaselineMLP, self).__init__()
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28 * 28, 64),
            nn.BatchNorm1d(64),
            nn.ReLU(inplace=True),
            nn.Dropout(0.2),
            nn.Linear(64, 10)
        )

    def forward(self, x):
        return self.classifier(x)
```

```
model_baseline = BaselineMLP().to(device)
param_count_baseline = sum(p.numel() for p in model_baseline.parameters() if p.requires_grad)
```

```
experimental_param_count = 4364554
print(f"Baseline MLP      - Total parameters: {param_count_baseline:,}")
print(f"ExperimentalMLP    - Total parameters: {experimental_param_count:,}")
print(f"Capacity ratio      - ExperimentalMLP is {experimental_param_count //
param_count_baseline}x larger")
```

```
Baseline MLP      - Total parameters: 51,018
ExperimentalMLP    - Total parameters: 4,364,554
Capacity ratio      - ExperimentalMLP is 85x larger
```

```
loss_criterion = nn.CrossEntropyLoss()
optimizer_baseline = optim.Adam(model_baseline.parameters(), lr=0.001, weight_decay=0.0)
```

```
BASELINE_EPOCHS = 12
```

```
print("Training baseline model...")
history_baseline = train_network(
    model=model_baseline,
    train_loader=train_loader,
    val_loader=val_loader,
    loss_fn=loss_criterion,
    opt=optimizer_baseline,
    epochs=BASELINE_EPOCHS,
    dev=device
)
```

```
Training baseline model...
```

```
{"model_id": "a021223b58004003a6f2b499d652107d", "version_major": 2, "version_minor": 0}
```

```
Epoch 01/12 | Train Loss: 0.4348 | Train Acc: 88.94% | Val Loss: 0.2176 | Val Acc: 93.81% | LR:
0.001000 | Time: 17.4s
```

```
{"model_id": "ea936f820ec544f58047238c0ace3533", "version_major": 2, "version_minor": 0}
```

Epoch 02/12 | Train Loss: 0.2313 | Train Acc: 93.23% | Val Loss: 0.1709 | Val Acc: 95.01% | LR: 0.001000 | Time: 18.0s

```
{"model_id": "54419756c39c4e979020971a057ff83a", "version_major": 2, "version_minor": 0}
```

Epoch 03/12 | Train Loss: 0.1942 | Train Acc: 94.17% | Val Loss: 0.1492 | Val Acc: 95.53% | LR: 0.001000 | Time: 16.8s

```
{"model_id": "98a6bb25742c487fbeeb4c88b8fffc1", "version_major": 2, "version_minor": 0}
```

Epoch 04/12 | Train Loss: 0.1707 | Train Acc: 94.88% | Val Loss: 0.1305 | Val Acc: 96.10% | LR: 0.001000 | Time: 15.5s

```
{"model_id": "52fa7b5c811a4463b4b88ecb1ee1a390", "version_major": 2, "version_minor": 0}
```

Epoch 05/12 | Train Loss: 0.1501 | Train Acc: 95.51% | Val Loss: 0.1271 | Val Acc: 96.08% | LR: 0.001000 | Time: 17.4s

```
{"model_id": "e8d80a2812834368aa9dcc00ca66cbee", "version_major": 2, "version_minor": 0}
```

Epoch 06/12 | Train Loss: 0.1419 | Train Acc: 95.71% | Val Loss: 0.1184 | Val Acc: 96.47% | LR: 0.001000 | Time: 17.1s

```
{"model_id": "470eaca6b7a4485a8b27b0136f54ccfb", "version_major": 2, "version_minor": 0}
```

Epoch 07/12 | Train Loss: 0.1278 | Train Acc: 95.95% | Val Loss: 0.1102 | Val Acc: 96.68% | LR: 0.001000 | Time: 17.3s

```
{"model_id": "1167c1b4cd0d4c099349c3bd7b0ca684", "version_major": 2, "version_minor": 0}
```

Epoch 08/12 | Train Loss: 0.1189 | Train Acc: 96.21% | Val Loss: 0.1091 | Val Acc: 96.83% | LR: 0.001000 | Time: 16.5s

```
{"model_id": "9c59fb07e21848da9a09a252805a8060", "version_major": 2, "version_minor": 0}
```

Epoch 09/12 | Train Loss: 0.1167 | Train Acc: 96.39% | Val Loss: 0.1027 | Val Acc: 96.91% | LR: 0.001000 | Time: 17.3s

```
{"model_id": "be68d99a14f34efd9e3cab140fe78e64", "version_major": 2, "version_minor": 0}
```

Epoch 10/12 | Train Loss: 0.1110 | Train Acc: 96.52% | Val Loss: 0.1036 | Val Acc: 96.91% | LR: 0.001000 | Time: 16.5s

```
{"model_id": "ed73858bd2af4e7d8f8d9f200327552c", "version_major": 2, "version_minor": 0}
```

Epoch 11/12 | Train Loss: 0.1044 | Train Acc: 96.63% | Val Loss: 0.0991 | Val Acc: 97.00% | LR: 0.001000 | Time: 16.5s

```
{"model_id": "69df70a637ba49fba4f9713f341bb4e3", "version_major": 2, "version_minor": 0}
```

Epoch 12/12 | Train Loss: 0.1004 | Train Acc: 96.81% | Val Loss: 0.0996 | Val Acc: 97.18% | LR: 0.001000 | Time: 17.8s

Training finished in 203.9 seconds

```
plot_metrics(history_baseline, "Baseline Model", train_color=COLOR_UNREG_TRAIN,
val_color=COLOR_UNREG_VAL)
```

```
t_acc_baseline = history_baseline['train_acc'][-1]
v_acc_baseline = history_baseline['val_acc'][-1]
t_loss_baseline = history_baseline['train_loss'][-1]
v_loss_baseline = history_baseline['val_loss'][-1]
```

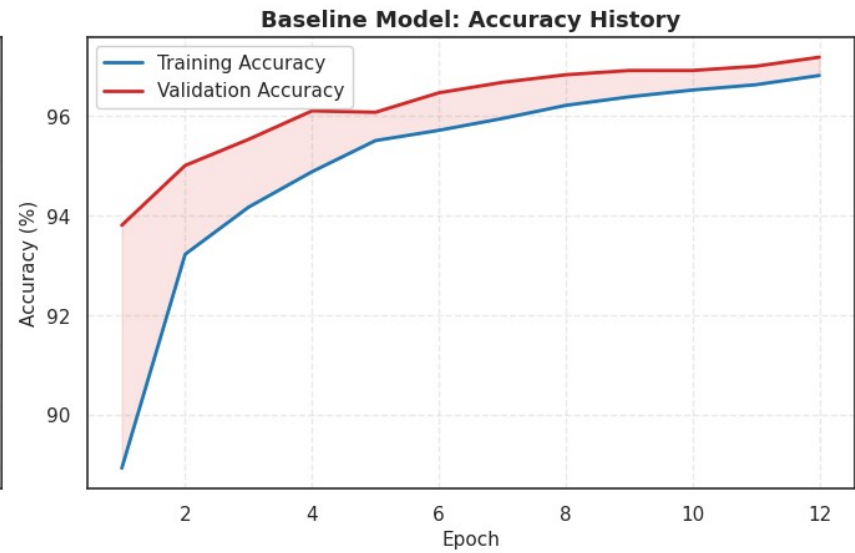
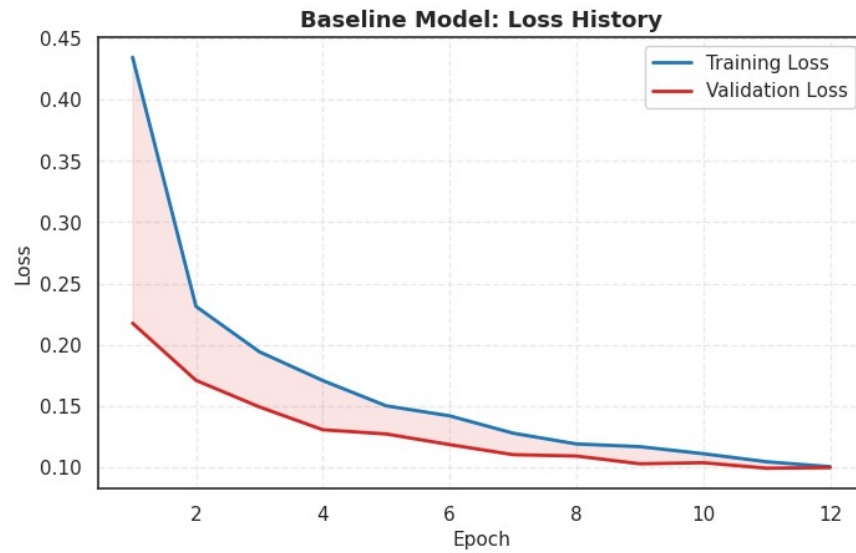
```
print(f"Final Training Accuracy: {t_acc_baseline:.2f}%")
```

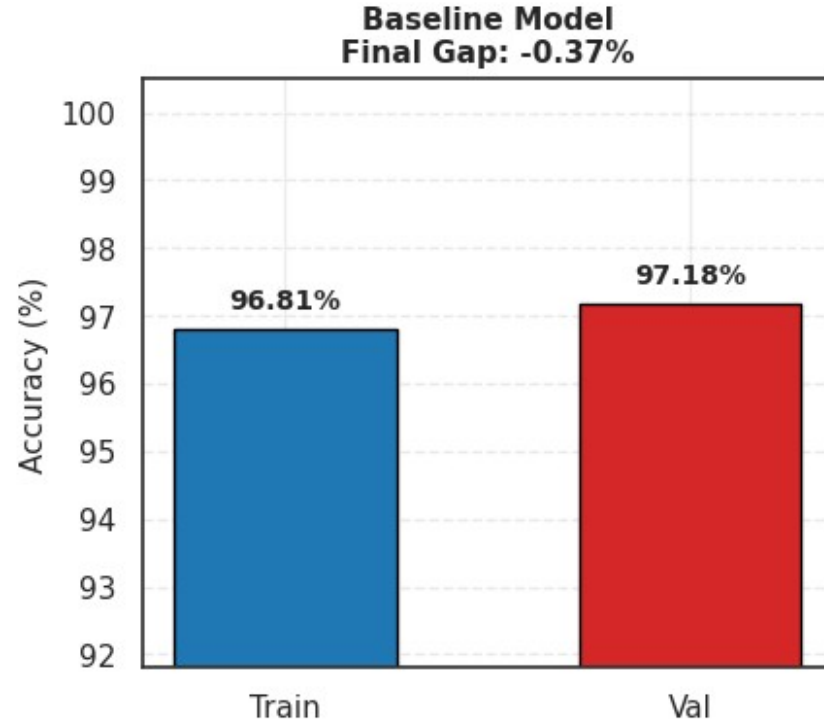
```

print(f"Final Validation Accuracy: {v_acc_baseline:.2f}%")
print(f"Difference: {t_acc_baseline - v_acc_baseline:.2f}%")
print(f"Final Training Loss: {t_loss_baseline:.4f}")
print(f"Final Validation Loss: {v_loss_baseline:.4f}")
print(f"Loss Ratio (Val / Train): {v_loss_baseline / max(t_loss_baseline, 1e-8):.2f}x")

```

Baseline Model -- Training Progress





Final Training Accuracy: 96.81%
Final Validation Accuracy: 97.18%
Difference: -0.37%
Final Training Loss: 0.1004
Final Validation Loss: 0.0996
Loss Ratio (Val / Train): 0.99x

6. Designing the Overfit Model

To study overfitting in a clear, unconstrained setting, we design a massive, fully-connected Multi-Layer Perceptron (MLP) with four hidden layers: the input image is flattened to a 784-dimensional vector and passed through the layers (2048 -> 1024 -> 512 -> 256 -> 10) in the class `ExperimentalMLP`.

An MLP treats every pixel as an independent feature and lacks convolutional spatial priors (such as weight sharing or spatial locality). Because it has no built-in architectural constraints, a high-capacity MLP is free to memorize exact pixel-level patterns specific to the training set. With over **4.36 million trainable parameters** (roughly 85 times the baseline), this model contains excessive capacity for simple 28x28 grayscale digit classification.

No dropout, batch normalization, or any other form of regularization is included in this architecture, since the explicit goal is to observe the model overfitting on the training data.

```
class ExperimentalMLP(nn.Module):  
    def __init__(self):  
        super(ExperimentalMLP, self).__init__()  
        self.classifier = nn.Sequential(  
            nn.Flatten(),  
            nn.Linear(28 * 28, 2048),  
            nn.ReLU(inplace=True),  
            nn.Linear(2048, 1024),  
            nn.ReLU(inplace=True),  
            nn.Linear(1024, 512),  
            nn.ReLU(inplace=True),  
            nn.Linear(512, 256),  
            nn.ReLU(inplace=True),  
            nn.Linear(256, 10)  
        )  
  
    def forward(self, x):  
        return self.classifier(x)  
  
test_model = ExperimentalMLP().to(device)  
param_count = sum(p.numel() for p in test_model.parameters() if p.requires_grad)  
print(f"Total parameters: {param_count:,}")  
print(f"Parameters per training sample: {param_count / len(train_set):.2f}")  
del test_model
```

```
Total parameters: 4,364,554  
Parameters per training sample: 89.07
```

7. Part A: Training Without Any Regularization

In this experiment, I trained the over-parameterized MLP (ExperimentalMLP, see Section 6) for 70 epochs on the full 49,000-image training dataset, using a batch size of 256 and learning rate 0.001. No weight decay is applied, and the training images are loaded without any data augmentation transforms.

Because this dense MLP architecture has no spatial constraints or regularization, we expect a pronounced generalization gap to emerge in both loss and accuracy as the model memorizes training noise. We track both the final-epoch numbers and the epoch at which the train-val accuracy gap is largest, and report both.

```
# Set up unregularized model
model_unreg = ExperimentalMLP().to(device)

optimizer_unreg = optim.Adam(model_unreg.parameters(), lr=0.001, weight_decay=0.0)

# Smaller batch size -> more gradient updates per epoch, noisier optimization,
# lets an over-parameterized net memorize training-set idiosyncrasies faster.
OVERFIT_BATCH_SIZE = 256
train_loader_unreg = DataLoader(train_set, batch_size=OVERFIT_BATCH_SIZE, shuffle=True,
num_workers=0)

# More epochs -> gives validation accuracy room to peak and then visibly
# degrade from its peak as the model keeps memorizing, rather than just plateau.
MAX_EPOCHS = 70

print("Training the unregularized model...")
history_unreg = train_network(
    model=model_unreg,
    train_loader=train_loader_unreg,
    val_loader=val_loader,
    loss_fn=loss_criterion,
    opt=optimizer_unreg,
    epochs=MAX_EPOCHS,
    dev=device
)
```

```
gap_per_epoch = [t - v for t, v in zip(history_unreg['train_acc'], history_unreg['val_acc'])]
peak_gap_epoch = int(np.argmax(gap_per_epoch))
print(f"\nPeak train-val accuracy gap: {gap_per_epoch[peak_gap_epoch]:.2f}% at epoch
{peak_gap_epoch + 1}")
print(f"Train acc at peak-gap epoch: {history_unreg['train_acc'][peak_gap_epoch]:.2f}%")
print(f"Val acc at peak-gap epoch: {history_unreg['val_acc'][peak_gap_epoch]:.2f}%")
```

Training the unregularized model...

```
{"model_id": "9d4802d60b3a44bbb57151d2ed6ce7d3", "version_major": 2, "version_minor": 0}
```

Epoch 01/70 | Train Loss: 0.2912 | Train Acc: 90.98% | Val Loss: 0.1473 | Val Acc: 95.59% | LR: 0.001000 | Time: 14.8s

```
{"model_id": "ecee8787c2154ed5825abd7b8680f4aa", "version_major": 2, "version_minor": 0}
```

Epoch 02/70 | Train Loss: 0.1010 | Train Acc: 96.89% | Val Loss: 0.1059 | Val Acc: 96.89% | LR: 0.001000 | Time: 14.7s

```
{"model_id": "2f55dfcc477247d2b9651dea5c04397a", "version_major": 2, "version_minor": 0}
```

Epoch 03/70 | Train Loss: 0.0674 | Train Acc: 97.91% | Val Loss: 0.0885 | Val Acc: 97.41% | LR: 0.001000 | Time: 14.6s

```
{"model_id": "d19bf13ec841499fa1d73f64da10915b", "version_major": 2, "version_minor": 0}
```

Epoch 04/70 | Train Loss: 0.0463 | Train Acc: 98.56% | Val Loss: 0.0976 | Val Acc: 97.05% | LR: 0.001000 | Time: 15.0s

```
{"model_id": "dc8631917b49488b99eb5393f73dc7dd", "version_major": 2, "version_minor": 0}
```

Epoch 05/70 | Train Loss: 0.0398 | Train Acc: 98.69% | Val Loss: 0.1006 | Val Acc: 97.53% | LR: 0.001000 | Time: 15.4s

```
{"model_id": "8ebf669c9d524ba0a1817a35311a48fb", "version_major": 2, "version_minor": 0}
```

Epoch 06/70 | Train Loss: 0.0363 | Train Acc: 98.83% | Val Loss: 0.1034 | Val Acc: 97.25% | LR: 0.001000 | Time: 15.2s

```
{"model_id": "7c71c0252bb14a0898bd60173530f303", "version_major": 2, "version_minor": 0}
```

Epoch 07/70 | Train Loss: 0.0289 | Train Acc: 99.08% | Val Loss: 0.0848 | Val Acc: 97.90% | LR: 0.001000 | Time: 15.2s

```
{"model_id": "2fb938019e7040559547755ac6b76a22", "version_major": 2, "version_minor": 0}
```

Epoch 08/70 | Train Loss: 0.0279 | Train Acc: 99.10% | Val Loss: 0.1178 | Val Acc: 97.18% | LR: 0.001000 | Time: 15.3s

```
{"model_id": "0501cbd77e3b4a308bf0fad92b0437fa", "version_major": 2, "version_minor": 0}
```

Epoch 09/70 | Train Loss: 0.0240 | Train Acc: 99.30% | Val Loss: 0.1106 | Val Acc: 97.39% | LR: 0.001000 | Time: 15.4s

```
{"model_id": "eb72f393f2df428095ce6ec96e799b1e", "version_major": 2, "version_minor": 0}
```

Epoch 10/70 | Train Loss: 0.0210 | Train Acc: 99.33% | Val Loss: 0.0859 | Val Acc: 97.93% | LR: 0.001000 | Time: 14.8s

```
{"model_id": "b64b6e056545420eb84c5aede793dbf5", "version_major": 2, "version_minor": 0}
```

Epoch 11/70 | Train Loss: 0.0174 | Train Acc: 99.47% | Val Loss: 0.0978 | Val Acc: 97.94% | LR: 0.001000 | Time: 14.7s

```
{"model_id": "cb77c849b63b447db0144f4575925199", "version_major": 2, "version_minor": 0}
```

Epoch 12/70 | Train Loss: 0.0168 | Train Acc: 99.48% | Val Loss: 0.1249 | Val Acc: 97.30% | LR: 0.001000 | Time: 15.2s

```
{"model_id": "6483f86c42e34ae38cad6f85e9534d92", "version_major": 2, "version_minor": 0}
```

Epoch 13/70 | Train Loss: 0.0186 | Train Acc: 99.43% | Val Loss: 0.1178 | Val Acc: 97.87% | LR: 0.001000 | Time: 14.9s

```
{"model_id": "ebfbc9d5f3b142b9a5cf8b2a23b2fd56", "version_major": 2, "version_minor": 0}
```

Epoch 14/70 | Train Loss: 0.0178 | Train Acc: 99.45% | Val Loss: 0.1072 | Val Acc: 97.83% | LR: 0.001000 | Time: 16.0s

```
{"model_id": "8b1341dec5864075b2506a78bf0e4967", "version_major": 2, "version_minor": 0}
```

Epoch 15/70 | Train Loss: 0.0147 | Train Acc: 99.53% | Val Loss: 0.0924 | Val Acc: 98.04% | LR: 0.001000 | Time: 15.5s

```
{"model_id": "8aa87ea3cc304b1897aa5e3e22cca0", "version_major": 2, "version_minor": 0}
```

Epoch 16/70 | Train Loss: 0.0111 | Train Acc: 99.67% | Val Loss: 0.0948 | Val Acc: 98.04% | LR: 0.001000 | Time: 14.7s

```
{"model_id": "937fbc3c7fd440cbb6ff8cc2ecbbbfef", "version_major": 2, "version_minor": 0}
```

Epoch 17/70 | Train Loss: 0.0118 | Train Acc: 99.67% | Val Loss: 0.1482 | Val Acc: 97.46% | LR: 0.001000 | Time: 14.3s

```
{"model_id": "b68508664cea430192f2b44e4c0361f8", "version_major": 2, "version_minor": 0}
```

Epoch 18/70 | Train Loss: 0.0127 | Train Acc: 99.64% | Val Loss: 0.0971 | Val Acc: 98.15% | LR: 0.001000 | Time: 15.5s

`{"model_id": "37ba9219a4bf49a08699e86192e7c12e", "version_major": 2, "version_minor": 0}`

Epoch 19/70 | Train Loss: 0.0131 | Train Acc: 99.63% | Val Loss: 0.1073 | Val Acc: 97.90% | LR: 0.001000 | Time: 15.0s

`{"model_id": "759a668a42dd46b8aaf2b7193605c718", "version_major": 2, "version_minor": 0}`

Epoch 20/70 | Train Loss: 0.0127 | Train Acc: 99.63% | Val Loss: 0.1066 | Val Acc: 98.10% | LR: 0.001000 | Time: 15.3s

`{"model_id": "d3fcc2fc00e443a992a6d12008d2933e", "version_major": 2, "version_minor": 0}`

Epoch 21/70 | Train Loss: 0.0122 | Train Acc: 99.66% | Val Loss: 0.1184 | Val Acc: 98.03% | LR: 0.001000 | Time: 15.0s

`{"model_id": "b80c4942a01546988060c5cf9bb26871", "version_major": 2, "version_minor": 0}`

Epoch 22/70 | Train Loss: 0.0110 | Train Acc: 99.69% | Val Loss: 0.1063 | Val Acc: 98.04% | LR: 0.001000 | Time: 15.0s

`{"model_id": "16d8184602a5449ba6da280ee2baa792", "version_major": 2, "version_minor": 0}`

Epoch 23/70 | Train Loss: 0.0139 | Train Acc: 99.62% | Val Loss: 0.1212 | Val Acc: 97.87% | LR: 0.001000 | Time: 14.0s

`{"model_id": "94f208d90f73443d994d6aebcd089ed8", "version_major": 2, "version_minor": 0}`

Epoch 24/70 | Train Loss: 0.0103 | Train Acc: 99.73% | Val Loss: 0.1382 | Val Acc: 97.75% | LR: 0.001000 | Time: 14.9s

{"model_id": "fa37502749874590916348d756781c94", "version_major": 2, "version_minor": 0}

Epoch 25/70 | Train Loss: 0.0154 | Train Acc: 99.60% | Val Loss: 0.0996 | Val Acc: 98.12% | LR: 0.001000 | Time: 15.3s

{"model_id": "fc0c0139e62c44b482cc00c59c22588f", "version_major": 2, "version_minor": 0}

Epoch 26/70 | Train Loss: 0.0112 | Train Acc: 99.70% | Val Loss: 0.0890 | Val Acc: 98.37% | LR: 0.001000 | Time: 15.0s

{"model_id": "a2a8b37837af4d868ee5481f3ec86e85", "version_major": 2, "version_minor": 0}

Epoch 27/70 | Train Loss: 0.0058 | Train Acc: 99.83% | Val Loss: 0.1184 | Val Acc: 98.01% | LR: 0.001000 | Time: 14.0s

{"model_id": "6cc25474ea78438896a18ffb7c343030", "version_major": 2, "version_minor": 0}

Epoch 28/70 | Train Loss: 0.0062 | Train Acc: 99.84% | Val Loss: 0.1400 | Val Acc: 97.70% | LR: 0.001000 | Time: 14.7s

{"model_id": "15e66dd5e2ff4742ad45ed12a2a33518", "version_major": 2, "version_minor": 0}

Epoch 29/70 | Train Loss: 0.0072 | Train Acc: 99.80% | Val Loss: 0.1306 | Val Acc: 97.78% | LR: 0.001000 | Time: 15.4s

{"model_id": "943ba2fe2b9f4151893182f63e88498f", "version_major": 2, "version_minor": 0}

Epoch 30/70 | Train Loss: 0.0084 | Train Acc: 99.79% | Val Loss: 0.1078 | Val Acc: 98.28% | LR: 0.001000 | Time: 14.7s

`{"model_id": "bcf221e278f64ce2ab8acf3ceb0f931f", "version_major": 2, "version_minor": 0}`

Epoch 31/70 | Train Loss: 0.0055 | Train Acc: 99.86% | Val Loss: 0.1281 | Val Acc: 98.03% | LR: 0.001000 | Time: 16.0s

`{"model_id": "360e08ad5801439aa002a9480605c880", "version_major": 2, "version_minor": 0}`

Epoch 32/70 | Train Loss: 0.0073 | Train Acc: 99.84% | Val Loss: 0.1367 | Val Acc: 98.08% | LR: 0.001000 | Time: 14.8s

`{"model_id": "abd5d80028414b678b14ffe93978f5b4", "version_major": 2, "version_minor": 0}`

Epoch 33/70 | Train Loss: 0.0059 | Train Acc: 99.84% | Val Loss: 0.1273 | Val Acc: 98.17% | LR: 0.001000 | Time: 14.2s

`{"model_id": "0c967147514e4932bbdf0837bacbe887", "version_major": 2, "version_minor": 0}`

Epoch 34/70 | Train Loss: 0.0033 | Train Acc: 99.92% | Val Loss: 0.1540 | Val Acc: 97.74% | LR: 0.001000 | Time: 14.2s

`{"model_id": "5b430a992be84188b2806042070f1b41", "version_major": 2, "version_minor": 0}`

Epoch 35/70 | Train Loss: 0.0094 | Train Acc: 99.77% | Val Loss: 0.1510 | Val Acc: 98.04% | LR: 0.001000 | Time: 15.6s

`{"model_id": "d9f23f2380304e9b8d9020a9274962fd", "version_major": 2, "version_minor": 0}`

Epoch 36/70 | Train Loss: 0.0092 | Train Acc: 99.74% | Val Loss: 0.1316 | Val Acc: 98.29% | LR: 0.001000 | Time: 15.2s

`{"model_id": "8ad24f29c7b141b49549929d10eb0eed", "version_major": 2, "version_minor": 0}`

Epoch 37/70 | Train Loss: 0.0092 | Train Acc: 99.75% | Val Loss: 0.1596 | Val Acc: 97.90% | LR: 0.001000 | Time: 16.0s

`{"model_id": "b0acaa98c2d34705810822cb7567d333", "version_major": 2, "version_minor": 0}`

Epoch 38/70 | Train Loss: 0.0078 | Train Acc: 99.86% | Val Loss: 0.1374 | Val Acc: 98.34% | LR: 0.001000 | Time: 15.2s

`{"model_id": "03d311eb519b4ebfa0dae1017f951e69", "version_major": 2, "version_minor": 0}`

Epoch 39/70 | Train Loss: 0.0082 | Train Acc: 99.80% | Val Loss: 0.1412 | Val Acc: 98.10% | LR: 0.001000 | Time: 14.4s

`{"model_id": "d5bf7c5139274885a9038a01a649cd50", "version_major": 2, "version_minor": 0}`

Epoch 40/70 | Train Loss: 0.0135 | Train Acc: 99.71% | Val Loss: 0.1245 | Val Acc: 97.95% | LR: 0.001000 | Time: 14.4s

`{"model_id": "330d0ea3843141c484917d68d0c9fca2", "version_major": 2, "version_minor": 0}`

Epoch 41/70 | Train Loss: 0.0079 | Train Acc: 99.77% | Val Loss: 0.1240 | Val Acc: 98.24% | LR: 0.001000 | Time: 14.4s

`{"model_id": "c9beba4f217d4675b2890240d6cb33af", "version_major": 2, "version_minor": 0}`

Epoch 42/70 | Train Loss: 0.0030 | Train Acc: 99.92% | Val Loss: 0.1333 | Val Acc: 98.01% | LR: 0.001000 | Time: 15.4s

`{"model_id": "72c3095a58364759a1bbfd3364d239ce", "version_major": 2, "version_minor": 0}`

Epoch 43/70 | Train Loss: 0.0059 | Train Acc: 99.84% | Val Loss: 0.1353 | Val Acc: 98.12% | LR: 0.001000 | Time: 15.0s

`{"model_id": "1997d9a08aff4b38b14169dde72f1bfb", "version_major": 2, "version_minor": 0}`

Epoch 44/70 | Train Loss: 0.0071 | Train Acc: 99.84% | Val Loss: 0.1275 | Val Acc: 98.03% | LR: 0.001000 | Time: 15.1s

`{"model_id": "1157153717394b259da6f857ec0244ca", "version_major": 2, "version_minor": 0}`

Epoch 45/70 | Train Loss: 0.0069 | Train Acc: 99.85% | Val Loss: 0.1419 | Val Acc: 98.30% | LR: 0.001000 | Time: 14.3s

`{"model_id": "ef0be269ef244bd8b577b77ad465d603", "version_major": 2, "version_minor": 0}`

Epoch 46/70 | Train Loss: 0.0061 | Train Acc: 99.86% | Val Loss: 0.1771 | Val Acc: 98.00% | LR: 0.001000 | Time: 15.4s

`{"model_id": "5a97370881dd4f6f9de41ec15ae7253a", "version_major": 2, "version_minor": 0}`

Epoch 47/70 | Train Loss: 0.0104 | Train Acc: 99.73% | Val Loss: 0.1393 | Val Acc: 98.13% | LR: 0.001000 | Time: 15.5s

`{"model_id": "e43a773e3b274157ba759a761d20f31b", "version_major": 2, "version_minor": 0}`

Epoch 48/70 | Train Loss: 0.0036 | Train Acc: 99.91% | Val Loss: 0.1440 | Val Acc: 98.31% | LR: 0.001000 | Time: 15.2s

{"model_id": "460df2b7da924e3fb55efbfb255f857d", "version_major": 2, "version_minor": 0}

Epoch 49/70 | Train Loss: 0.0057 | Train Acc: 99.84% | Val Loss: 0.1296 | Val Acc: 98.18% | LR: 0.001000 | Time: 15.0s

{"model_id": "bdfeafe50cf0413ca00c76a1b0648123", "version_major": 2, "version_minor": 0}

Epoch 50/70 | Train Loss: 0.0026 | Train Acc: 99.92% | Val Loss: 0.1420 | Val Acc: 98.27% | LR: 0.001000 | Time: 15.3s

{"model_id": "236276a4ddeb4386ada30d1500e3e5c2", "version_major": 2, "version_minor": 0}

Epoch 51/70 | Train Loss: 0.0052 | Train Acc: 99.89% | Val Loss: 0.1400 | Val Acc: 98.34% | LR: 0.001000 | Time: 15.2s

{"model_id": "4f17464968924b73bde3afee808c4d7c", "version_major": 2, "version_minor": 0}

Epoch 52/70 | Train Loss: 0.0011 | Train Acc: 99.98% | Val Loss: 0.1358 | Val Acc: 98.51% | LR: 0.001000 | Time: 14.8s

{"model_id": "ebff275f643641c9bc41cd2aa199a839", "version_major": 2, "version_minor": 0}

Epoch 53/70 | Train Loss: 0.0042 | Train Acc: 99.90% | Val Loss: 0.1287 | Val Acc: 98.38% | LR: 0.001000 | Time: 15.7s

{"model_id": "7c3c6540f92949b08b5dc2989c940caf", "version_major": 2, "version_minor": 0}

Epoch 54/70 | Train Loss: 0.0073 | Train Acc: 99.83% | Val Loss: 0.1546 | Val Acc: 97.99% | LR: 0.001000 | Time: 14.9s

`{"model_id": "d5f7082fa10844478ac389eef2d33744", "version_major": 2, "version_minor": 0}`

Epoch 55/70 | Train Loss: 0.0107 | Train Acc: 99.76% | Val Loss: 0.1325 | Val Acc: 97.75% | LR: 0.001000 | Time: 15.2s

`{"model_id": "42dc21ccdb7f4026a897ff04e1132064", "version_major": 2, "version_minor": 0}`

Epoch 56/70 | Train Loss: 0.0093 | Train Acc: 99.78% | Val Loss: 0.1470 | Val Acc: 97.90% | LR: 0.001000 | Time: 15.8s

`{"model_id": "be827f6dfa514ad393965a96a05blada", "version_major": 2, "version_minor": 0}`

Epoch 57/70 | Train Loss: 0.0122 | Train Acc: 99.73% | Val Loss: 0.1278 | Val Acc: 98.18% | LR: 0.001000 | Time: 15.8s

`{"model_id": "f2699003c84646eda01905211192adce", "version_major": 2, "version_minor": 0}`

Epoch 58/70 | Train Loss: 0.0055 | Train Acc: 99.85% | Val Loss: 0.1144 | Val Acc: 98.25% | LR: 0.001000 | Time: 15.4s

`{"model_id": "e7ce8d2d21d147448dd41845e9d76ccb", "version_major": 2, "version_minor": 0}`

Epoch 59/70 | Train Loss: 0.0028 | Train Acc: 99.96% | Val Loss: 0.1371 | Val Acc: 98.42% | LR: 0.001000 | Time: 15.0s

`{"model_id": "c3746bf7d21644ba9f28cf78eff476c9", "version_major": 2, "version_minor": 0}`

Epoch 60/70 | Train Loss: 0.0008 | Train Acc: 99.99% | Val Loss: 0.1757 | Val Acc: 98.36% | LR: 0.001000 | Time: 15.2s

```
{"model_id": "ea79ae50ce8c4c989fd14a70f5665e1e", "version_major": 2, "version_minor": 0}
```

Epoch 61/70 | Train Loss: 0.0027 | Train Acc: 99.93% | Val Loss: 0.1596 | Val Acc: 98.30% | LR: 0.001000 | Time: 15.0s

```
{"model_id": "5855091ec5b64e43a195e570b25d6add", "version_major": 2, "version_minor": 0}
```

Epoch 62/70 | Train Loss: 0.0081 | Train Acc: 99.84% | Val Loss: 0.1276 | Val Acc: 98.14% | LR: 0.001000 | Time: 15.4s

```
{"model_id": "726ef029a87e4429802aa2730fdb36b5", "version_major": 2, "version_minor": 0}
```

Epoch 63/70 | Train Loss: 0.0063 | Train Acc: 99.84% | Val Loss: 0.1445 | Val Acc: 98.29% | LR: 0.001000 | Time: 15.4s

```
{"model_id": "0062ee271eb64fd6bd9581ee09ecf226", "version_major": 2, "version_minor": 0}
```

Epoch 64/70 | Train Loss: 0.0046 | Train Acc: 99.89% | Val Loss: 0.1673 | Val Acc: 98.13% | LR: 0.001000 | Time: 15.4s

```
{"model_id": "860572bbc1f340c982119fcbb5e44c1", "version_major": 2, "version_minor": 0}
```

Epoch 65/70 | Train Loss: 0.0078 | Train Acc: 99.83% | Val Loss: 0.1357 | Val Acc: 98.06% | LR: 0.001000 | Time: 15.0s

```
{"model_id": "5dd8f9c6b4324f0fb294f46ae0db7724", "version_major": 2, "version_minor": 0}
```

Epoch 66/70 | Train Loss: 0.0038 | Train Acc: 99.90% | Val Loss: 0.1614 | Val Acc: 98.22% | LR: 0.001000 | Time: 14.6s

```
{"model_id": "e0fc236ff82b4853857ed090141bf87d", "version_major": 2, "version_minor": 0}
```

Epoch 67/70 | Train Loss: 0.0025 | Train Acc: 99.94% | Val Loss: 0.1602 | Val Acc: 98.41% | LR: 0.001000 | Time: 15.6s

```
{"model_id": "f560e9b9c5214f4f8035c2e57a5d062b", "version_major": 2, "version_minor": 0}
```

Epoch 68/70 | Train Loss: 0.0011 | Train Acc: 99.98% | Val Loss: 0.1544 | Val Acc: 98.36% | LR: 0.001000 | Time: 15.4s

```
{"model_id": "2223e521671c44f582cadfcb8bcfe26e", "version_major": 2, "version_minor": 0}
```

Epoch 69/70 | Train Loss: 0.0012 | Train Acc: 99.97% | Val Loss: 0.1527 | Val Acc: 98.38% | LR: 0.001000 | Time: 15.0s

```
{"model_id": "0ca138c01c1d490d9908d647da29d582", "version_major": 2, "version_minor": 0}
```

Epoch 70/70 | Train Loss: 0.0009 | Train Acc: 99.98% | Val Loss: 0.1952 | Val Acc: 98.40% | LR: 0.001000 | Time: 14.3s

Training finished in 1054.4 seconds

Peak train-val accuracy gap: 2.21% at epoch 17

Train acc at peak-gap epoch: 99.67%

Val acc at peak-gap epoch: 97.46%

Why a Dense MLP is Highly Susceptible to Overfitting

Unlike Convolutional architectures, a Multi-Layer Perceptron (MLP) has no built-in geometric constraints:

- **No Spatial Locality:** Every one of the 784 input pixels is connected directly to every unit in the first hidden layer. Two pixels that are adjacent in the image are treated exactly the same as two pixels on opposite corners.
- **No Weight Sharing:** Each connection has its own independent weight, which drastically increases the total parameter count and makes it easy for the network to find solutions that fit training noise perfectly.
- **No Translation Invariance:** If a stroke or digit shifts slightly, the MLP cannot recognize it without having trained on that exact shifted pixel layout.

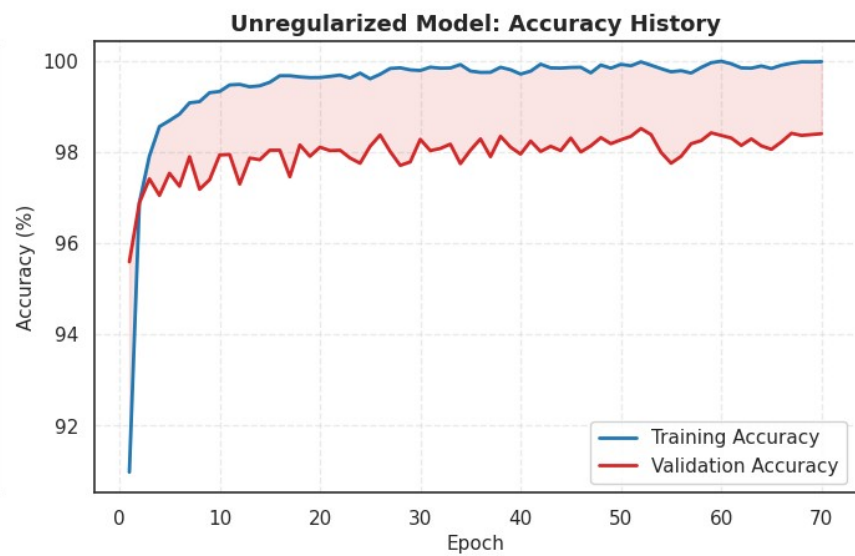
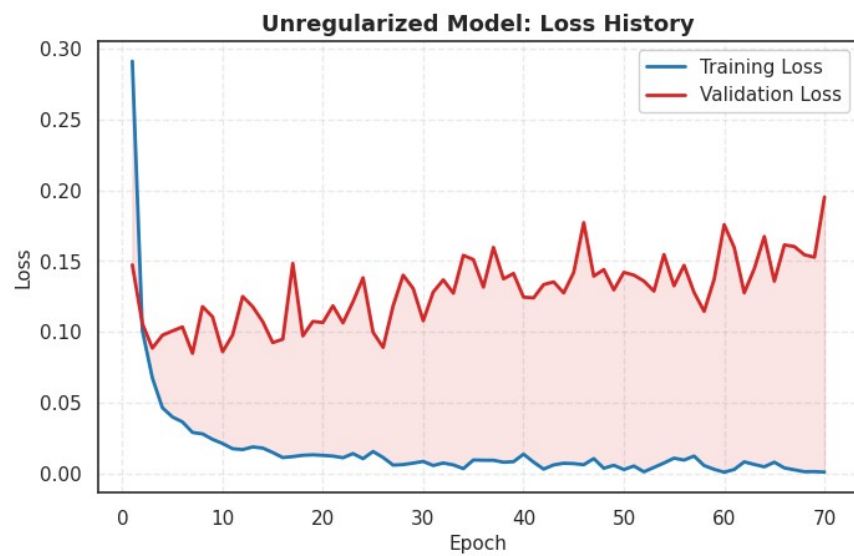
Without weight sharing or spatial priors, a large MLP is free to map arbitrary pixel configurations to class labels. This absence of inductive bias is the reason the unregularized model is expected to overfit severely on the training set, showing a wide generalization gap in both accuracy and validation loss.

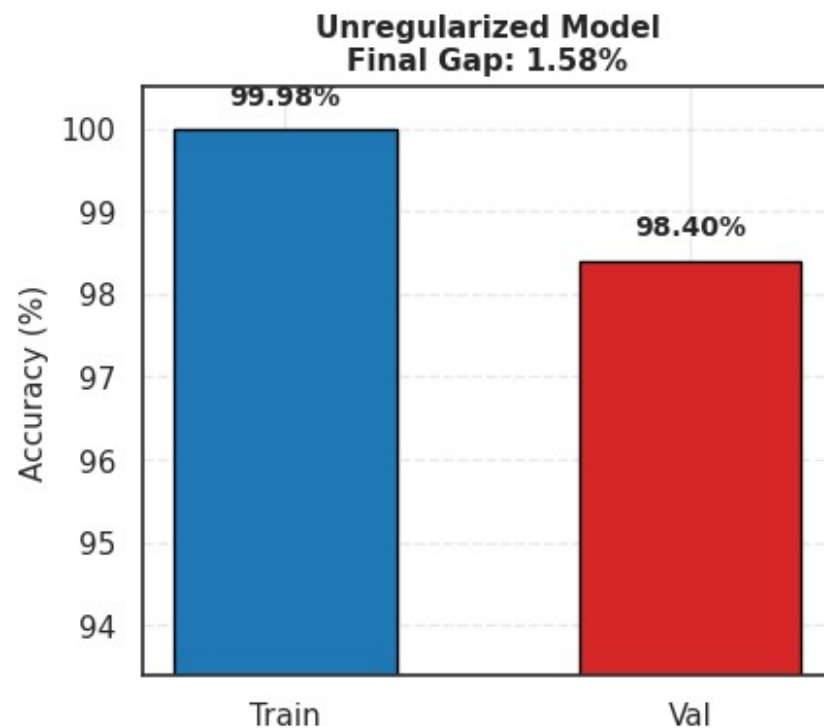
```
plot_metrics(history_unreg, "Unregularized Model", train_color=COLOR_UNREG_TRAIN,
val_color=COLOR_UNREG_VAL)

t_acc_final = history_unreg['train_acc'][-1]
v_acc_final = history_unreg['val_acc'][-1]
t_loss_final = history_unreg['train_loss'][-1]
v_loss_final = history_unreg['val_loss'][-1]

print(f"Final Training Accuracy: {t_acc_final:.2f}%")
print(f"Final Validation Accuracy: {v_acc_final:.2f}%")
print(f"Difference: {t_acc_final - v_acc_final:.2f}%")
print(f"Final Training Loss: {t_loss_final:.4f}")
print(f"Final Validation Loss: {v_loss_final:.4f}")
print(f"Loss Ratio (Val / Train): {v_loss_final / max(t_loss_final, 1e-8):.2f}x")
```

Unregularized Model -- Training Progress





Final Training Accuracy: 99.98%
Final Validation Accuracy: 98.40%
Difference: 1.58%
Final Training Loss: 0.0009
Final Validation Loss: 0.1952
Loss Ratio (Val / Train): 224.58x

8. Part B: Training With Regularization

To prevent the model from memorizing the data, I trained the **same MLP architecture** (unchanged from Part A) using the following training-only additions:

1. **Data Augmentation:** Random rotations (up to 15 degrees) and slight translations/scaling, applied to the image before it gets flattened into the MLP's input vector. This still prevents the network from memorizing exact pixel layouts, even though the network itself has no spatial structure.
2. **Weight Decay:** Added a decay value of $1e-3$ to the Adam optimizer to keep parameter values smaller.
3. **Early Stopping:** Stop training if the validation loss does not improve for 5 consecutive epochs.
4. **Learning Rate Scheduling:** Reduce the learning rate by half if validation loss plateaus for 2 epochs.

Both the unregularized model (Part A) and the regularized model (Part B) are trained on the full 49,000-image training dataset, using the identical MLP class (no architecture changes between Part A and Part B, only the training protocol changes).

Note on Batch Size Differences

To optimize training dynamics and curves:

- **Baseline Model:** Trained with batch size 64 (well-matched to the small model size).
- **Unregularized Model:** Trained with batch size 256 (helps smooth and stabilize gradients over 70 epochs).
- **Regularized Model:** Trained with batch size 128 (standard regularization training setup).

```
# Define transformations with augmentation
augmented_transform = transforms.Compose([
    transforms.RandomRotation(degrees=15),
    transforms.RandomAffine(degrees=0, translate=(0.1, 0.1), scale=(0.9, 1.1)),
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

# Create augmented training dataset from the combined pool
train_dataset_aug = datasets.MNIST(root='./data', train=True, download=True,
transform=augmented_transform)
test_dataset_aug = datasets.MNIST(root='./data', train=False, download=True,
transform=augmented_transform)
full_dataset_aug = ConcatDataset([train_dataset_aug, test_dataset_aug])

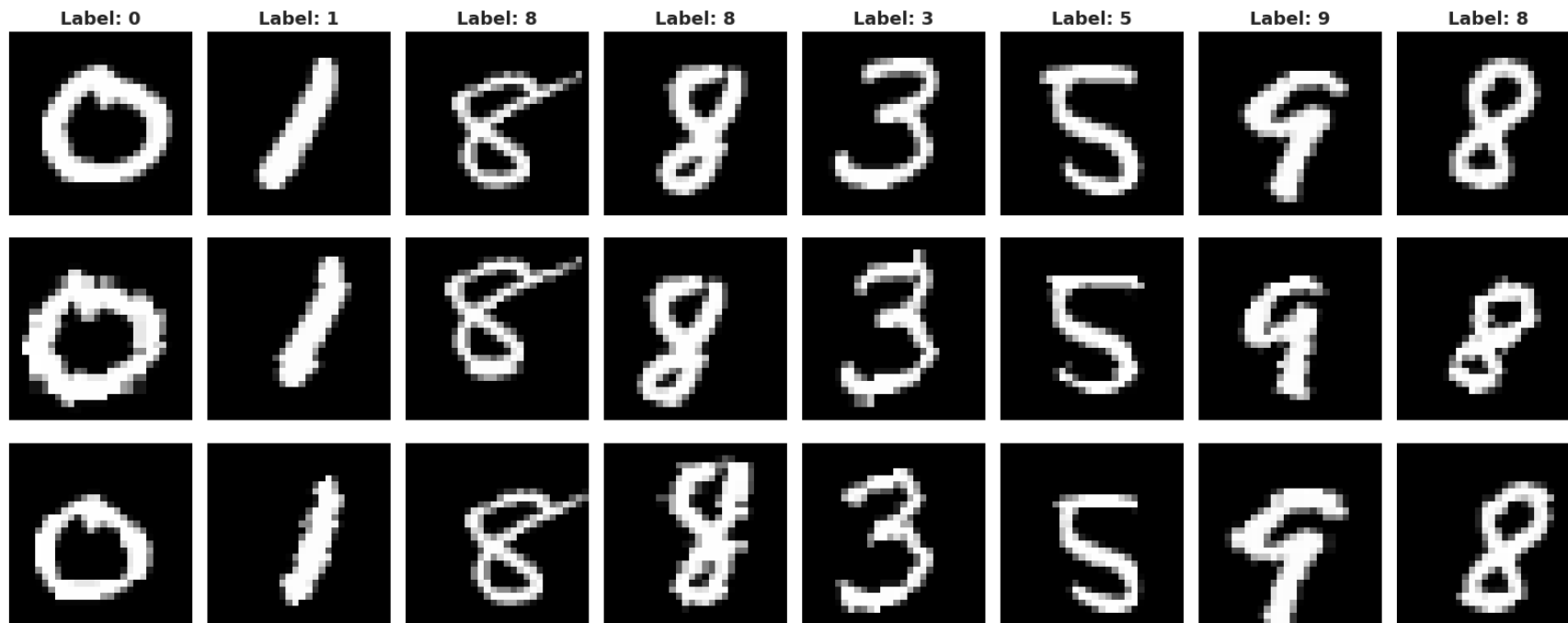
full_train_set_aug = Subset(full_dataset_aug, train_idx)
overfit_train_loader_aug = DataLoader(full_train_set_aug, batch_size=128, shuffle=True,
num_workers=0, pin_memory=True)
```

```

# Plot comparison
fig, axes = plt.subplots(3, 8, figsize=(16, 7))
for i in range(8):
    base_img, label = train_set[i]
    axes[0, i].imshow((base_img.squeeze() * 0.3081 + 0.1307).numpy(), cmap='gray')
    axes[0, i].set_title(f"Label: {label}")
    axes[0, i].axis('off')
    aug_img_1, _ = full_train_set_aug[i]
    axes[1, i].imshow((aug_img_1.squeeze() * 0.3081 + 0.1307).numpy(), cmap='gray')
    axes[1, i].axis('off')
    aug_img_2, _ = full_train_set_aug[i]
    axes[2, i].imshow((aug_img_2.squeeze() * 0.3081 + 0.1307).numpy(), cmap='gray')
    axes[2, i].axis('off')
axes[0, 0].set_ylabel("Original", rotation=0, labelpad=40, fontsize=12, fontweight='bold')
axes[1, 0].set_ylabel("Augmented 1", rotation=0, labelpad=40, fontsize=12, fontweight='bold')
axes[2, 0].set_ylabel("Augmented 2", rotation=0, labelpad=40, fontsize=12, fontweight='bold')
plt.suptitle("Comparison of Original and Augmented Images", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

```

Comparison of Original and Augmented Images



```
# Set up regularized model
model_reg = ExperimentalMLP().to(device)

optimizer_reg = optim.Adam(model_reg.parameters(), lr=0.001, weight_decay=1e-3)

# Set up learning rate scheduler
scheduler_reg = optim.lr_scheduler.ReduceLRonPlateau(
    optimizer_reg, mode='min', factor=0.5, patience=2
)

REG_EPOCHS = 30
EARLY_STOP_PATIENCE = 5
```

```
print("Training the regularized model...")
history_reg = train_network(
    model=model_reg,
    train_loader=overfit_train_loader_aug,
    val_loader=val_loader,
    loss_fn=loss_criterion,
    opt=optimizer_reg,
    epochs=REG_EPOCHS,
    dev=device,
    scheduler=scheduler_reg,
    early_stop_patience=EARLY_STOP_PATIENCE
)
```

Training the regularized model...

```
{"model_id": "557391ba41fb4929828911d0b9e9cd1b", "version_major": 2, "version_minor": 0}
```

Epoch 01/30 | Train Loss: 0.5196 | Train Acc: 83.04% | Val Loss: 0.1329 | Val Acc: 96.08% | LR: 0.001000 | Time: 25.0s

```
{"model_id": "0217de52614b45aaa761f43e52869e4b", "version_major": 2, "version_minor": 0}
```

Epoch 02/30 | Train Loss: 0.2673 | Train Acc: 91.69% | Val Loss: 0.1291 | Val Acc: 96.14% | LR: 0.001000 | Time: 25.8s

```
{"model_id": "b3607d420f0f4938aeea38a0d8f578b1", "version_major": 2, "version_minor": 0}
```

Epoch 03/30 | Train Loss: 0.2262 | Train Acc: 92.89% | Val Loss: 0.1091 | Val Acc: 96.77% | LR: 0.001000 | Time: 26.1s

```
{"model_id": "0b1cd777d62b4348899550126bbbed631", "version_major": 2, "version_minor": 0}
```

Epoch 04/30 | Train Loss: 0.2049 | Train Acc: 93.48% | Val Loss: 0.0830 | Val Acc: 97.46% | LR: 0.001000 | Time: 26.0s

```
{"model_id": "62b0625857de4ac0b9a5b0eb5a0cdda9", "version_major": 2, "version_minor": 0}
```

Epoch 05/30 | Train Loss: 0.1889 | Train Acc: 94.15% | Val Loss: 0.0987 | Val Acc: 96.92% | LR: 0.001000 | Time: 25.1s

```
{"model_id": "e71f7ba0076d4a62b1f032b2934c81dd", "version_major": 2, "version_minor": 0}
```

Epoch 06/30 | Train Loss: 0.1817 | Train Acc: 94.29% | Val Loss: 0.0773 | Val Acc: 97.48% | LR: 0.001000 | Time: 26.0s

```
{"model_id": "2acf0e7e98d947e9ade38ebcd62a07e7", "version_major": 2, "version_minor": 0}
```

Epoch 07/30 | Train Loss: 0.1740 | Train Acc: 94.50% | Val Loss: 0.0821 | Val Acc: 97.53% | LR: 0.001000 | Time: 25.9s

```
{"model_id": "0cb77a7175ae4d72ae2df32467d07808", "version_major": 2, "version_minor": 0}
```

Epoch 08/30 | Train Loss: 0.1650 | Train Acc: 94.83% | Val Loss: 0.0841 | Val Acc: 97.44% | LR: 0.001000 | Time: 25.3s

```
{"model_id": "23751345f6ac4652b08c2c067fc42f50", "version_major": 2, "version_minor": 0}
```

Epoch 09/30 | Train Loss: 0.1541 | Train Acc: 95.12% | Val Loss: 0.0860 | Val Acc: 97.53% | LR: 0.001000 | Time: 26.2s

```
{"model_id": "ee713d0628394306afb3dfb454549292", "version_major": 2, "version_minor": 0}
```

Epoch 10/30 | Train Loss: 0.1207 | Train Acc: 96.22% | Val Loss: 0.0581 | Val Acc: 98.34% | LR: 0.000500 | Time: 25.1s

```
{"model_id": "61bc99e7eebb4fafa7d749bd1863735b", "version_major": 2, "version_minor": 0}
```

Epoch 11/30 | Train Loss: 0.1115 | Train Acc: 96.58% | Val Loss: 0.0557 | Val Acc: 98.21% | LR: 0.000500 | Time: 25.4s

`{"model_id": "486f137c68bf4e02b0f7a2a82e81cef8", "version_major": 2, "version_minor": 0}`

Epoch 12/30 | Train Loss: 0.1120 | Train Acc: 96.54% | Val Loss: 0.0572 | Val Acc: 98.26% | LR: 0.000500 | Time: 26.3s

`{"model_id": "2eb69c60993c4ab5bdc745f31e7b4c04", "version_major": 2, "version_minor": 0}`

Epoch 13/30 | Train Loss: 0.1098 | Train Acc: 96.49% | Val Loss: 0.0557 | Val Acc: 98.31% | LR: 0.000500 | Time: 27.2s

`{"model_id": "b3ba971e62cf477086312e08d412d50d", "version_major": 2, "version_minor": 0}`

Epoch 14/30 | Train Loss: 0.1105 | Train Acc: 96.54% | Val Loss: 0.0569 | Val Acc: 98.31% | LR: 0.000500 | Time: 25.7s

`{"model_id": "a32b40cb3caa4144b7eca21977e25296", "version_major": 2, "version_minor": 0}`

Epoch 15/30 | Train Loss: 0.0939 | Train Acc: 97.04% | Val Loss: 0.0469 | Val Acc: 98.54% | LR: 0.000250 | Time: 26.4s

`{"model_id": "28d02af7491540869750831830fdd8cc", "version_major": 2, "version_minor": 0}`

Epoch 16/30 | Train Loss: 0.0911 | Train Acc: 97.16% | Val Loss: 0.0497 | Val Acc: 98.47% | LR: 0.000250 | Time: 26.2s

`{"model_id": "1b3361bfa6b0400baeaf8fe98289eb4c", "version_major": 2, "version_minor": 0}`

Epoch 17/30 | Train Loss: 0.0860 | Train Acc: 97.30% | Val Loss: 0.0438 | Val Acc: 98.65% | LR: 0.000250 | Time: 25.7s

`{"model_id": "be344852a14c4574941bd83d7761e506", "version_major": 2, "version_minor": 0}`

Epoch 18/30 | Train Loss: 0.0897 | Train Acc: 97.15% | Val Loss: 0.0429 | Val Acc: 98.74% | LR: 0.000250 | Time: 26.2s

`{"model_id": "51a275ae1ed44021bab778bec3513554", "version_major": 2, "version_minor": 0}`

Epoch 19/30 | Train Loss: 0.0872 | Train Acc: 97.28% | Val Loss: 0.0402 | Val Acc: 98.86% | LR: 0.000250 | Time: 25.8s

`{"model_id": "46c04d2ed9414ee3bde87276eed9664d", "version_major": 2, "version_minor": 0}`

Epoch 20/30 | Train Loss: 0.0881 | Train Acc: 97.26% | Val Loss: 0.0441 | Val Acc: 98.75% | LR: 0.000250 | Time: 25.3s

`{"model_id": "0e53aa30e7b54091b14702653de571eb", "version_major": 2, "version_minor": 0}`

Epoch 21/30 | Train Loss: 0.0871 | Train Acc: 97.29% | Val Loss: 0.0443 | Val Acc: 98.50% | LR: 0.000250 | Time: 25.3s

`{"model_id": "ceeea3a012114662bf9ddcd92da960f2", "version_major": 2, "version_minor": 0}`

Epoch 22/30 | Train Loss: 0.0840 | Train Acc: 97.37% | Val Loss: 0.0395 | Val Acc: 98.87% | LR: 0.000250 | Time: 25.3s

`{"model_id": "297b65d963544875bdc43f56d144adbb", "version_major": 2, "version_minor": 0}`

Epoch 23/30 | Train Loss: 0.0848 | Train Acc: 97.37% | Val Loss: 0.0455 | Val Acc: 98.70% | LR: 0.000250 | Time: 25.7s

`{"model_id": "a243ec5f1f8c44e1b56fd7181a9dee5c", "version_major": 2, "version_minor": 0}`

Epoch 24/30 | Train Loss: 0.0852 | Train Acc: 97.32% | Val Loss: 0.0430 | Val Acc: 98.76% | LR: 0.000250 | Time: 25.5s

`{"model_id": "8b0bd12f45294514845eefedd3100008", "version_major": 2, "version_minor": 0}`

Epoch 25/30 | Train Loss: 0.0832 | Train Acc: 97.31% | Val Loss: 0.0402 | Val Acc: 98.90% | LR: 0.000250 | Time: 25.1s

`{"model_id": "5276289599244117bbba6634275eae6a", "version_major": 2, "version_minor": 0}`

Epoch 26/30 | Train Loss: 0.0739 | Train Acc: 97.72% | Val Loss: 0.0374 | Val Acc: 98.91% | LR: 0.000125 | Time: 24.7s

`{"model_id": "6a8d3cc8f4dc47f7b5bc7b6fe5720f42", "version_major": 2, "version_minor": 0}`

Epoch 27/30 | Train Loss: 0.0709 | Train Acc: 97.80% | Val Loss: 0.0357 | Val Acc: 99.00% | LR: 0.000125 | Time: 25.6s

`{"model_id": "7ae07152f4374069bf4736bd8092db0e", "version_major": 2, "version_minor": 0}`

Epoch 28/30 | Train Loss: 0.0710 | Train Acc: 97.76% | Val Loss: 0.0386 | Val Acc: 98.88% | LR: 0.000125 | Time: 25.9s

`{"model_id": "3ea96c989e304ffeb6ec53ad3c789e77", "version_major": 2, "version_minor": 0}`

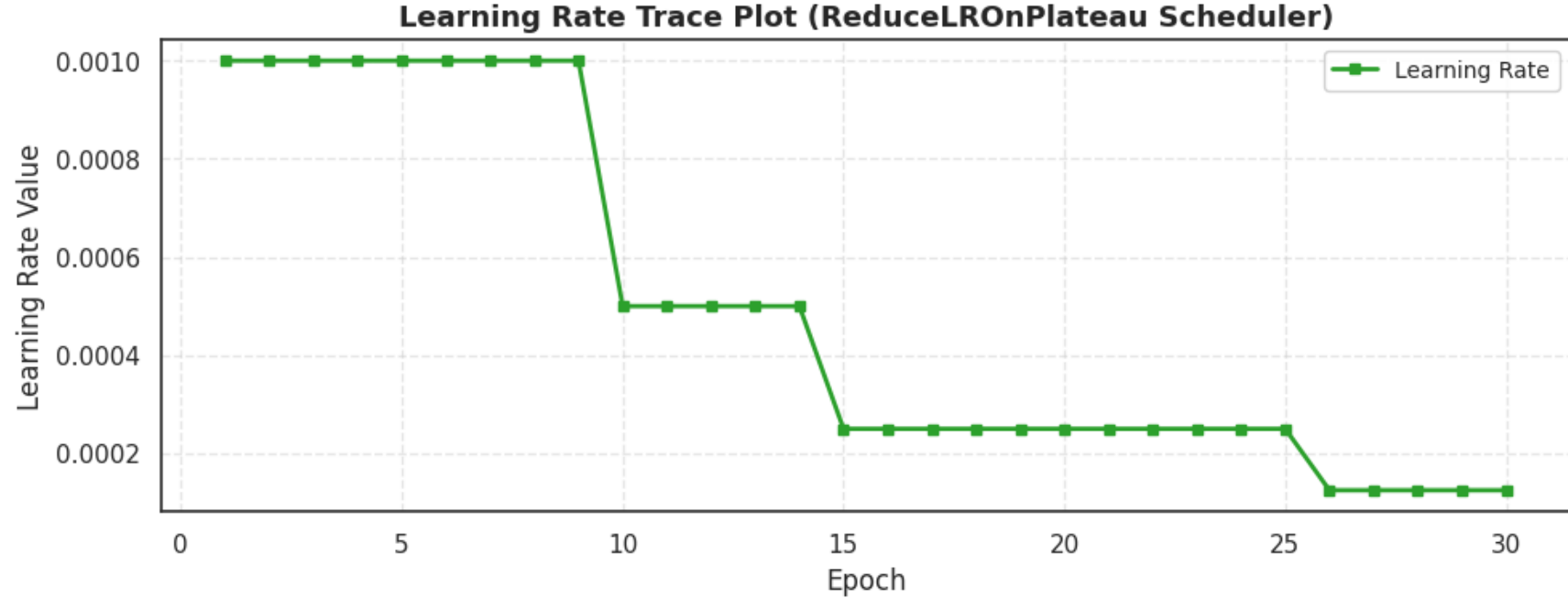
Epoch 29/30 | Train Loss: 0.0695 | Train Acc: 97.86% | Val Loss: 0.0342 | Val Acc: 99.05% | LR: 0.000125 | Time: 25.9s

```
{"model_id": "98a89835dc134a0d8797e2d0002cc232", "version_major": 2, "version_minor": 0}
```

Epoch 30/30 | Train Loss: 0.0705 | Train Acc: 97.82% | Val Loss: 0.0360 | Val Acc: 98.92% | LR: 0.000125 | Time: 25.8s
Training finished in 771.6 seconds

8.1. Learning Rate Scheduler Behavior Analysis

```
plt.figure(figsize=(10, 4))
epochs_reg_list = range(1, len(history_reg['learning_rates']) + 1)
plt.plot(epochs_reg_list, history_reg['learning_rates'], color='#2ca02c', marker='s',
linewidth=2, label='Learning Rate')
plt.xlabel('Epoch', fontsize=12)
plt.ylabel('Learning Rate Value', fontsize=12)
plt.title('Learning Rate Trace Plot (ReduceLROnPlateau Scheduler)', fontsize=13,
fontweight='bold')
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(fontsize=10)
plt.tight_layout()
plt.show()
```

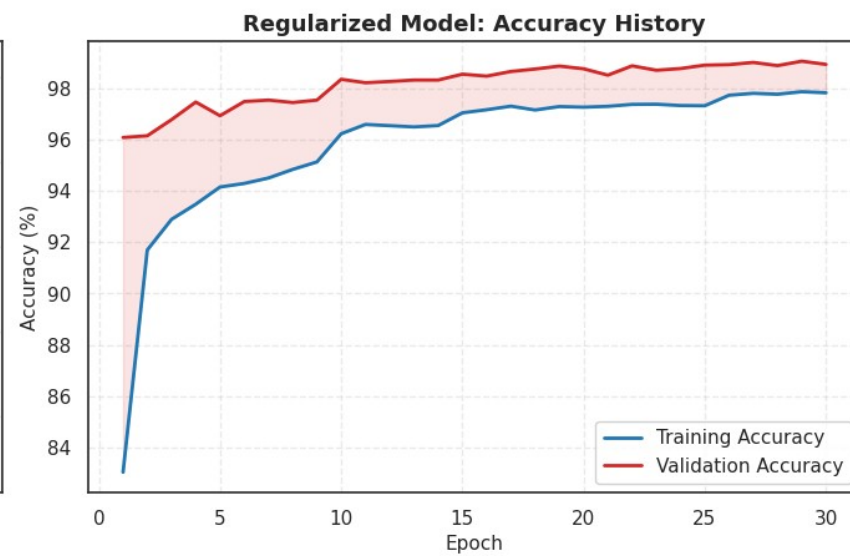


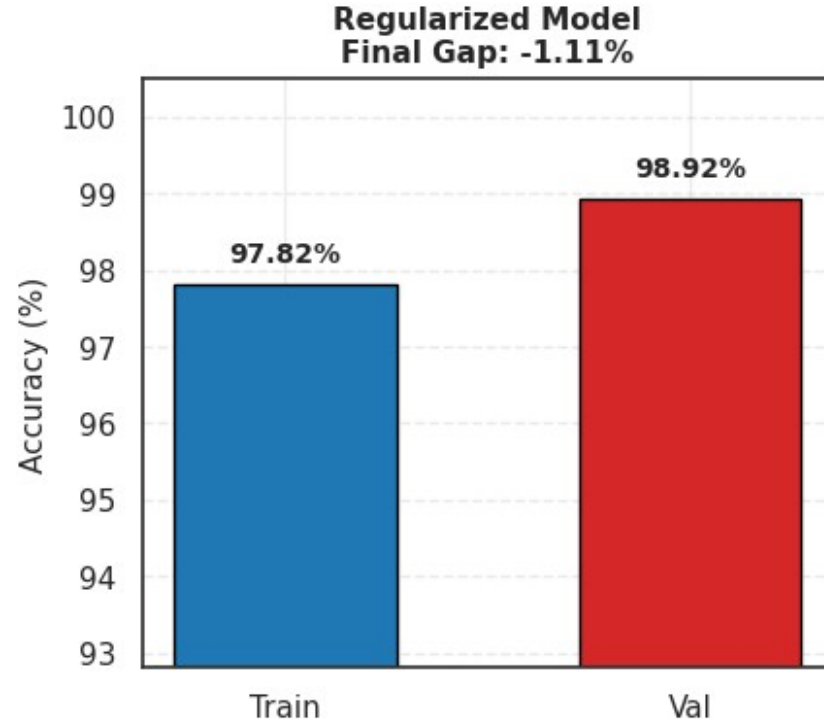
```
plot_metrics(history_reg, "Regularized Model", train_color=COLOR_UNREG_TRAIN,
val_color=COLOR_UNREG_VAL)

t_acc_reg_final = history_reg['train_acc'][-1]
v_acc_reg_final = history_reg['val_acc'][-1]
t_loss_reg_final = history_reg['train_loss'][-1]
v_loss_reg_final = history_reg['val_loss'][-1]

print(f"Final Training Accuracy: {t_acc_reg_final:.2f}%")
print(f"Final Validation Accuracy: {v_acc_reg_final:.2f}%")
print(f"Difference: {t_acc_reg_final - v_acc_reg_final:.2f}%")
print(f"Final Training Loss: {t_loss_reg_final:.4f}")
print(f"Final Validation Loss: {v_loss_reg_final:.4f}")
print(f"Loss Ratio (Val / Train): {v_loss_reg_final / max(t_loss_reg_final, 1e-8):.2f}x")
```

Regularized Model -- Training Progress





```
Final Training Accuracy: 97.82%  
Final Validation Accuracy: 98.92%  
Difference: -1.11%  
Final Training Loss: 0.0705  
Final Validation Loss: 0.0360  
Loss Ratio (Val / Train): 0.51x
```

9. Direct Training History Comparison

```
fig, axes = plt.subplots(2, 2, figsize=(16, 12))  
  
ep_base = range(1, len(history_baseline['train_loss']) + 1)
```

```

ep_unreg = range(1, len(history_unreg['train_loss']) + 1)
ep_reg   = range(1, len(history_reg['train_loss']) + 1)

# Loss Comparison
axes[0, 0].plot(ep_base, history_baseline['train_loss'], color=COLOR_BASELINE_TRAIN,
linestyle=':', linewidth=2, label='Baseline: Train')
axes[0, 0].plot(ep_base, history_baseline['val_loss'], color=COLOR_BASELINE_VAL,
linestyle=':', linewidth=2, label='Baseline: Val')
axes[0, 0].plot(ep_unreg, history_unreg['train_loss'], color=COLOR_UNREG_TRAIN,
linestyle='--', label='Unregularized: Train')
axes[0, 0].plot(ep_unreg, history_unreg['val_loss'], color=COLOR_UNREG_VAL,
linestyle='--', label='Unregularized: Val')
axes[0, 0].plot(ep_reg, history_reg['train_loss'], color=COLOR_REG_TRAIN,
linestyle='-', linewidth=2, label='Regularized: Train')
axes[0, 0].plot(ep_reg, history_reg['val_loss'], color=COLOR_REG_VAL,
linestyle='-', linewidth=2, label='Regularized: Val')
axes[0, 0].set_xlabel('Epoch'); axes[0, 0].set_ylabel('Loss')
axes[0, 0].set_title('Loss History Comparison')
axes[0, 0].legend(fontsize=8); axes[0, 0].grid(True, alpha=0.3)

# Accuracy Comparison
axes[0, 1].plot(ep_base, history_baseline['train_acc'], color=COLOR_BASELINE_TRAIN,
linestyle=':', linewidth=2, label='Baseline: Train')
axes[0, 1].plot(ep_base, history_baseline['val_acc'], color=COLOR_BASELINE_VAL,
linestyle=':', linewidth=2, label='Baseline: Val')
axes[0, 1].plot(ep_unreg, history_unreg['train_acc'], color=COLOR_UNREG_TRAIN,
linestyle='--', label='Unregularized: Train')
axes[0, 1].plot(ep_unreg, history_unreg['val_acc'], color=COLOR_UNREG_VAL,
linestyle='--', label='Unregularized: Val')
axes[0, 1].plot(ep_reg, history_reg['train_acc'], color=COLOR_REG_TRAIN,
linestyle='-', linewidth=2, label='Regularized: Train')
axes[0, 1].plot(ep_reg, history_reg['val_acc'], color=COLOR_REG_VAL,
linestyle='-', linewidth=2, label='Regularized: Val')
axes[0, 1].set_xlabel('Epoch'); axes[0, 1].set_ylabel('Accuracy (%)')
axes[0, 1].set_title('Accuracy History Comparison')

```

```

axes[0, 1].legend(fontsize=8); axes[0, 1].grid(True, alpha=0.3)

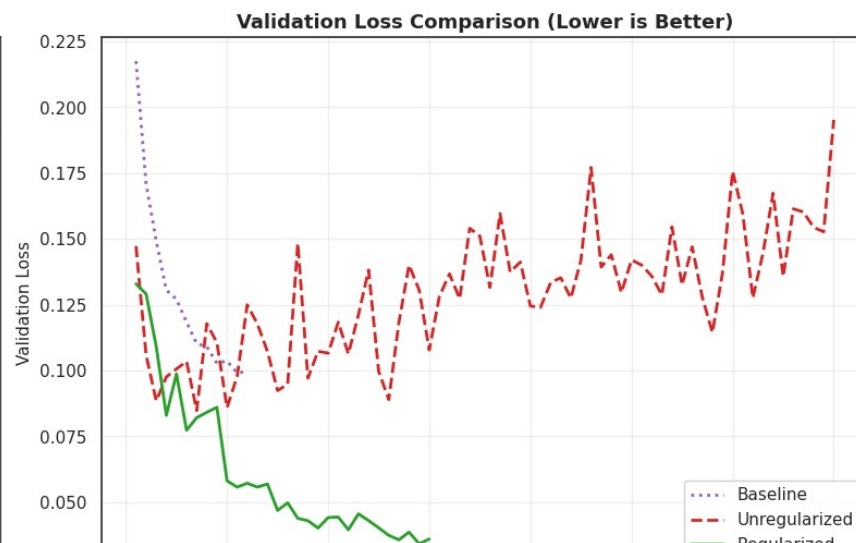
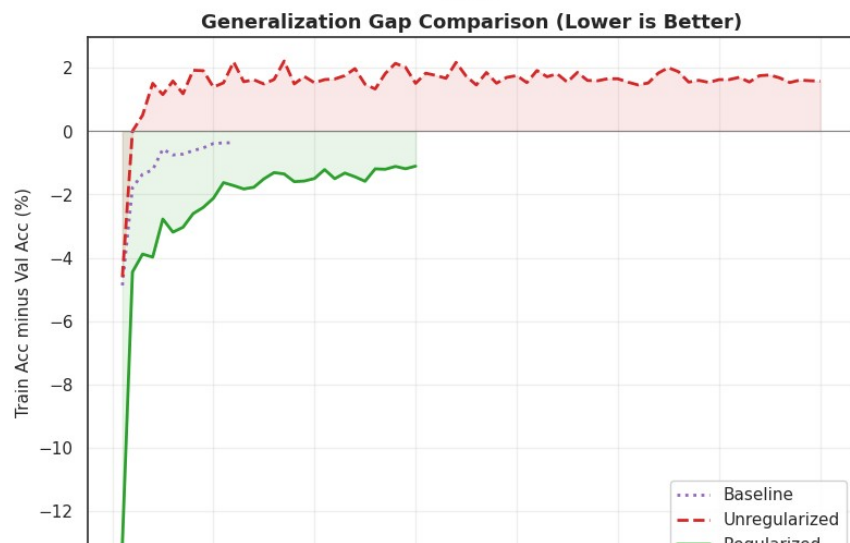
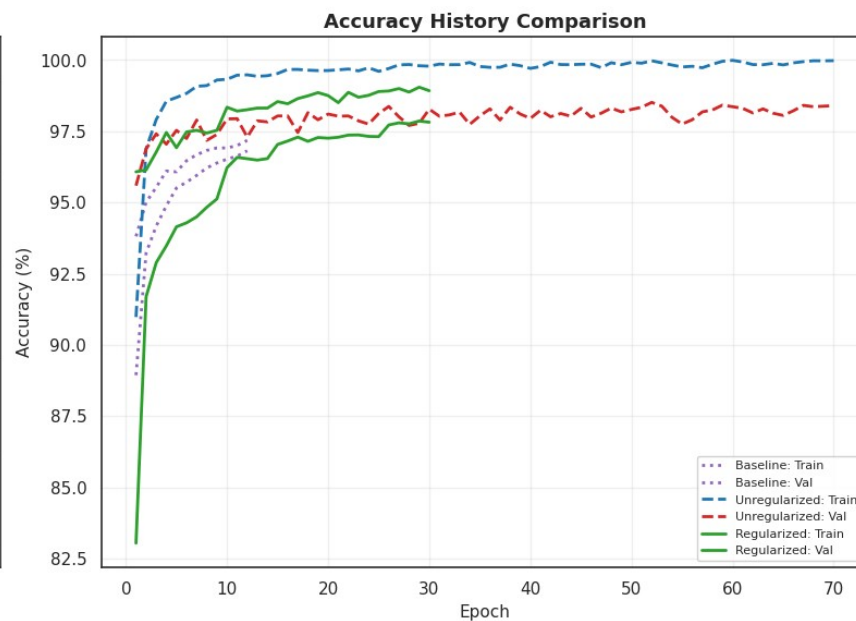
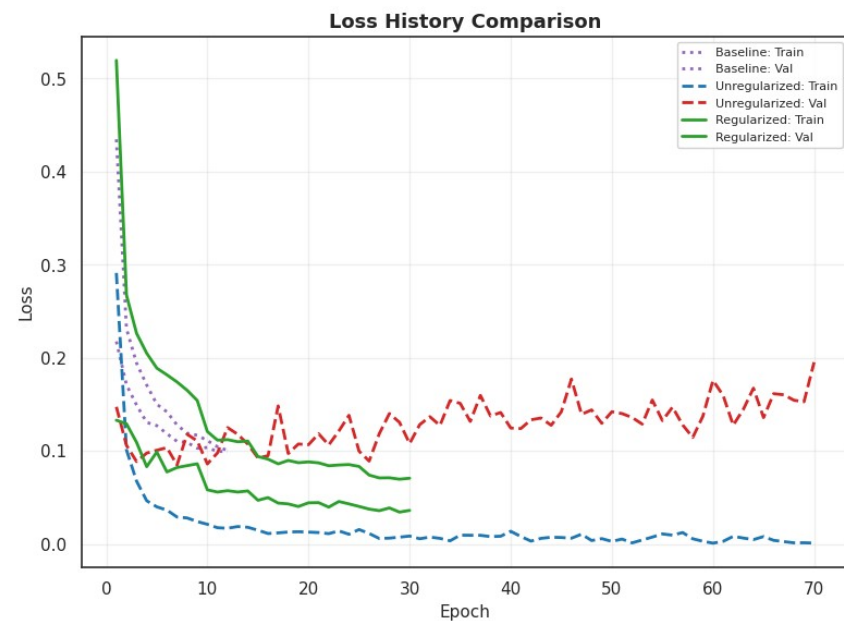
# Generalization Gap
gap_baseline = [t - v for t, v in zip(history_baseline['train_acc'],
history_baseline['val_acc'])]
gap_unreg     = [t - v for t, v in zip(history_unreg['train_acc'],   history_unreg['val_acc'])]
gap_reg       = [t - v for t, v in zip(history_reg['train_acc'],   history_reg['val_acc'])]
axes[1, 0].plot(ep_base, gap_baseline, color=COLOR_BASELINE_TRAIN, linestyle=':', linewidth=2,
label='Baseline')
axes[1, 0].plot(ep_unreg, gap_unreg,    color=COLOR_UNREG_VAL,      linestyle='--',
label='Unregularized')
axes[1, 0].plot(ep_reg, gap_reg,        color=COLOR_REG_TRAIN,     linestyle='-', linewidth=2,
label='Regularized')
axes[1, 0].fill_between(ep_unreg, gap_unreg, alpha=0.1, color=COLOR_UNREG_VAL)
axes[1, 0].fill_between(ep_reg, gap_reg, alpha=0.1, color=COLOR_REG_TRAIN)
axes[1, 0].axhline(y=0, color='gray', linestyle='-', linewidth=0.8)
axes[1, 0].set_xlabel('Epoch'); axes[1, 0].set_ylabel('Train Acc minus Val Acc (%)')
axes[1, 0].set_title('Generalization Gap Comparison (Lower is Better)')
axes[1, 0].legend(); axes[1, 0].grid(True, alpha=0.3)

# Validation Loss Comparison
axes[1, 1].plot(ep_base, history_baseline['val_loss'], color=COLOR_BASELINE_TRAIN,
linestyle=':', linewidth=2, label='Baseline')
axes[1, 1].plot(ep_unreg, history_unreg['val_loss'], color=COLOR_UNREG_VAL,
linestyle='--', label='Unregularized')
axes[1, 1].plot(ep_reg, history_reg['val_loss'], color=COLOR_REG_TRAIN, linewidth=2, label='Regularized')
axes[1, 1].set_xlabel('Epoch'); axes[1, 1].set_ylabel('Validation Loss')
axes[1, 1].set_title('Validation Loss Comparison (Lower is Better)')
axes[1, 1].legend(); axes[1, 1].grid(True, alpha=0.3)

plt.suptitle('Comparison of All Three Runs', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

```

Comparison of All Three Runs



Generalization Gap Plot Interpretation

Note that in the early epochs, the gap for both models may appear negative. This occurs because the model's validation accuracy temporarily exceeds its training accuracy while the optimizer is still in the rapid learning phase. This behavior normalizes as training progresses and the model begins to saturate on the training data.

The **Generalization Gap** graph tracks the difference between training accuracy and validation accuracy over time. Interpreting these curves provides insights into the models' training behavior:

- **Baseline (purple dotted):** The gap stays small and near zero throughout, even with no regularization. This is the expected behavior when model capacity matches task complexity.
- **A Growing Gap (Unregularized Model):** The red curve shows a gap that continually expands over the training duration. As the training accuracy approaches 100%, the validation accuracy lags behind or plateaus. This increasing distance indicates that the model is memorizing training-specific details (overfitting) rather than learning features that generalize well.
- **A Stable, Near-Zero Gap (Regularized Model):** The green curve shows a gap that remains small and stable. This suggests that the regularization constraints (data augmentation, weight decay, early stopping) are working as intended, preventing the model from fitting noise and keeping validation performance aligned with training performance.
- **What to Compare:** When looking at the three curves, compare the final value of the gaps and the shape of their trajectories. A trajectory that diverges indicates unchecked overfitting, whereas a trajectory that remains flat demonstrates stable learning.

10. Test Set Performance Evaluation

```
test_loss_baseline, test_acc_baseline = evaluate_performance(model_baseline, test_loader,
loss_criterion, device)
test_loss_unreg, test_acc_unreg = evaluate_performance(model_unreg, test_loader,
loss_criterion, device)
test_loss_reg, test_acc_reg = evaluate_performance(model_reg, test_loader,
loss_criterion, device)

print("Evaluation Results on Test Set:")
print("=" * 70)
print(f"Baseline Model - Test Accuracy: {test_acc_baseline:.2f}% | Test Loss:
{test_loss_baseline:.4f}")
print(f"Unregularized Model - Test Accuracy: {test_acc_unreg:.2f}% | Test Loss:
{test_loss_unreg:.4f}")
```

```

print(f"Regularized Model    - Test Accuracy: {test_acc_reg:.2f}% | Test Loss: {test_loss_reg:.4f}")
print("=" * 70)
print(f"Train to Test Gap (Baseline):      {t_acc_baseline - test_acc_baseline:.2f}%")
print(f"Train to Test Gap (Unregularized): {t_acc_final      - test_acc_unreg:.2f}%")
print(f"Train to Test Gap (Regularized):    {t_acc_reg_final  - test_acc_reg:.2f}%")

```

Evaluation Results on Test Set:

```

=====
Baseline Model      - Test Accuracy: 96.86% | Test Loss: 0.0970
Unregularized Model - Test Accuracy: 98.39% | Test Loss: 0.1905
Regularized Model   - Test Accuracy: 98.84% | Test Loss: 0.0361
=====
Train to Test Gap (Baseline):      -0.04%
Train to Test Gap (Unregularized): 1.59%
Train to Test Gap (Regularized):   -1.02%

```

Test Set Performance Comparison Plot

The table above gives exact numbers; this chart puts the three models' final test loss and test accuracy side by side so the differences are easier to see at a glance.

```

# Bar-chart comparison of final test-set performance across all three models
models_cmp = ['Baseline', 'Unregularized', 'Regularized']
test_acc_cmp = [test_acc_baseline, test_acc_unreg, test_acc_reg]
test_loss_cmp = [test_loss_baseline, test_loss_unreg, test_loss_reg]
colors_cmp = ['#9467bd', '#d62728', '#2ca02c']

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

bars1 = ax1.bar(models_cmp, test_loss_cmp, color=colors_cmp, edgecolor='black')
ax1.set_ylabel('Test Loss')
ax1.set_title('Test Loss by Model (Lower is Better)')
ax1.grid(True, axis='y', linestyle='--', alpha=0.5)
for b, v in zip(bars1, test_loss_cmp):

```

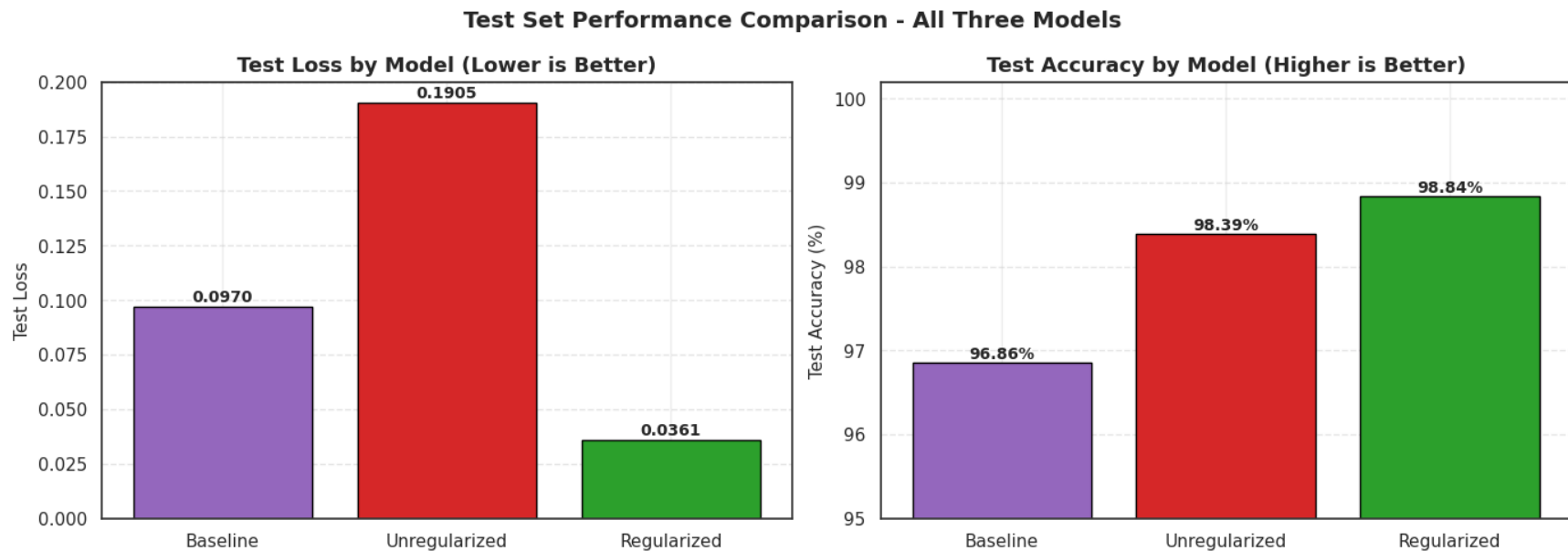
```

    ax1.text(b.get_x() + b.get_width()/2, v + 0.002, f'{v:.4f}', ha='center', fontsize=10,
fontweight='bold')

bars2 = ax2.bar(models_cmp, test_acc_cmp, color=colors_cmp, edgecolor='black')
ax2.set_ylabel('Test Accuracy (%)')
ax2.set_title('Test Accuracy by Model (Higher is Better)')
ax2.set_ylim(95, 100.2)
ax2.grid(True, axis='y', linestyle='--', alpha=0.5)
for b, v in zip(bars2, test_acc_cmp):
    ax2.text(b.get_x() + b.get_width()/2, v + 0.05, f'{v:.2f}%', ha='center', fontsize=10,
fontweight='bold')

plt.suptitle('Test Set Performance Comparison - All Three Models', fontsize=14,
fontweight='bold')
plt.tight_layout()
plt.show()

```



Summary Comparison Table

```
acc_gap_baseline = t_acc_baseline - v_acc_baseline
acc_gap_unreg    = t_acc_final    - v_acc_final
acc_gap_reg      = t_acc_reg_final - v_acc_reg_final

print("Model Performance Summary Table")
print("=" * 115)
print(f"{'Model Name':<25} | {'Final Train Acc':<15} | {'Final Val Acc':<15} | {'Test Acc':<12} | {'Train-Val Gap':<15} | {'Test Loss':<10}")
print("-" * 115)
print(f"{'Baseline Model':<25} | {t_acc_baseline:>13.2f}% | {v_acc_baseline:>13.2f}% | {test_acc_baseline:>10.2f}% | {acc_gap_baseline:>13.2f}% | {test_loss_baseline:>9.4f}")
print(f"{'Unregularized Model':<25} | {t_acc_final:>13.2f}% | {v_acc_final:>13.2f}% | {test_acc_unreg:>10.2f}% | {acc_gap_unreg:>13.2f}% | {test_loss_unreg:>9.4f}")
print(f"{'Regularized Model':<25} | {t_acc_reg_final:>13.2f}% | {v_acc_reg_final:>13.2f}% | {test_acc_reg:>10.2f}% | {acc_gap_reg:>13.2f}% | {test_loss_reg:>9.4f}")
print("=" * 115)
```

Model Performance Summary Table

=====					
=====					
Model Name	Final Train Acc	Final Val Acc	Test Acc	Train-Val Gap	Test Loss

Baseline Model	96.81%	97.18%	96.86%	-0.37%	0.0970
Unregularized Model	99.98%	98.40%	98.39%	1.58%	0.1905
Regularized Model	97.82%	98.92%	98.84%	-1.11%	0.0361
=====					
=====					

Confusion Matrices

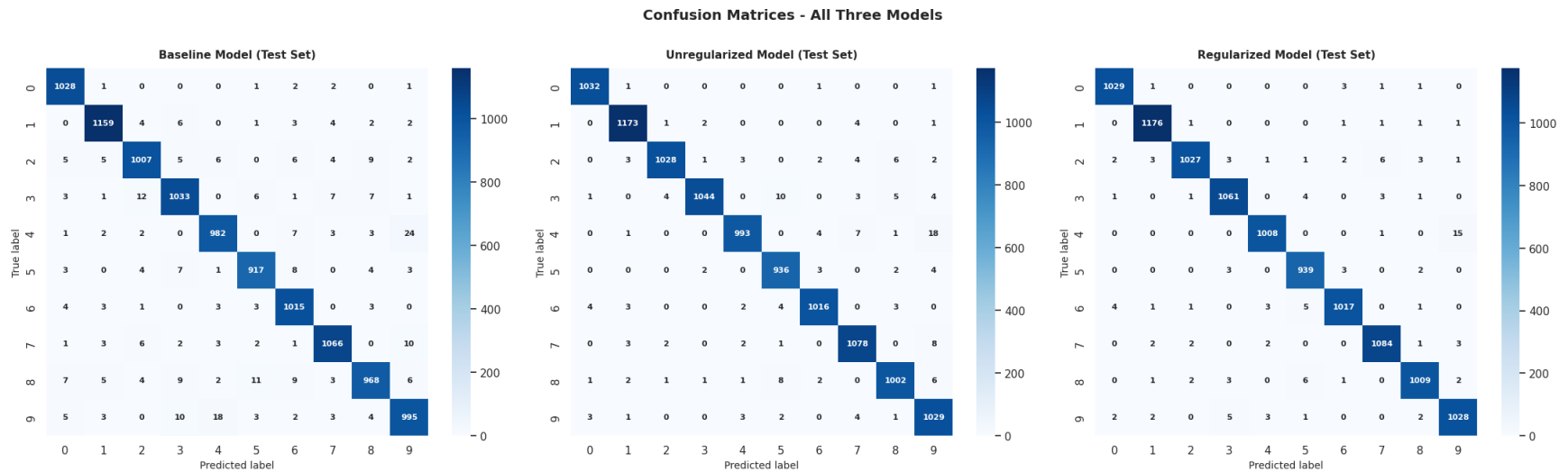
```
def extract_all_predictions(model, loader, dev):
    model.eval()
    preds_list = []
    labels_list = []
    with torch.no_grad():
        for inputs, targets in loader:
            inputs = inputs.to(dev)
            outputs = model(inputs)
            _, preds = outputs.max(1)
            preds_list.extend(preds.cpu().numpy())
            labels_list.extend(targets.numpy())
    return np.array(labels_list), np.array(preds_list)

labels_baseline, preds_baseline = extract_all_predictions(model_baseline, test_loader, device)
labels_unreg, preds_unreg = extract_all_predictions(model_unreg, test_loader, device)
labels_reg, preds_reg = extract_all_predictions(model_reg, test_loader, device)

cm_baseline = confusion_matrix(labels_baseline, preds_baseline)
cm_unreg = confusion_matrix(labels_unreg, preds_unreg)
cm_reg = confusion_matrix(labels_reg, preds_reg)

# Three confusion matrices side by side
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(22, 6.5))
for ax, cm, title in [
    (ax1, cm_baseline, "Baseline Model (Test Set)"),
    (ax2, cm_unreg, "Unregularized Model (Test Set)"),
    (ax3, cm_reg, "Regularized Model (Test Set)"),
]:
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
                cbar=True, annot_kws={"size": 8, "weight": "bold"})
    ax.set_title(title, fontsize=11, fontweight='bold', pad=10)
    ax.set_ylabel("True label", fontsize=10)
    ax.set_xlabel("Predicted label", fontsize=10)
plt.suptitle("Confusion Matrices - All Three Models", fontsize=14, fontweight='bold', y=1.01)
```

```
plt.tight_layout()
plt.show()
```



Per-Class Accuracy Analysis

```
def compute_class_accuracies(labels, preds):
    cm = confusion_matrix(labels, preds)
    class_totals = cm.sum(axis=1)
    class_correct = np.diag(cm)
    return 100.0 * class_correct / class_totals

acc_baseline = compute_class_accuracies(labels_baseline, preds_baseline)
acc_unreg = compute_class_accuracies(labels_unreg, preds_unreg)
acc_reg = compute_class_accuracies(labels_reg, preds_reg)

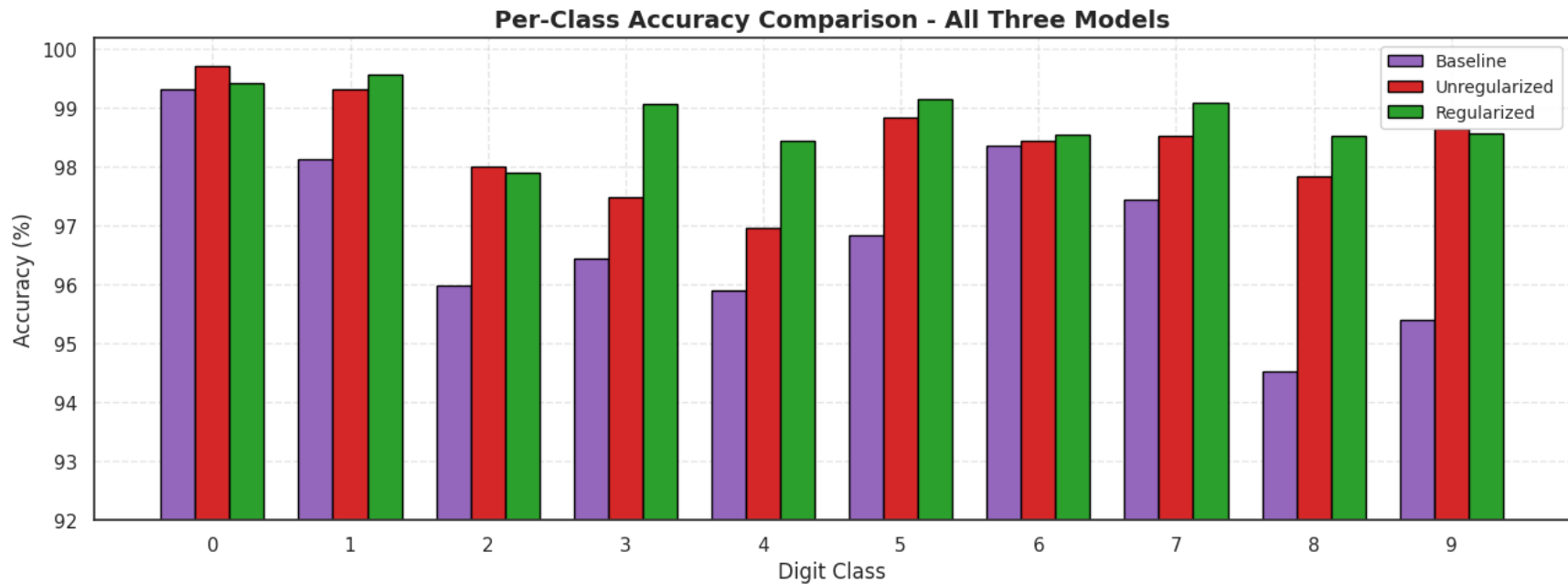
x = np.arange(10)
width = 0.25

fig, ax = plt.subplots(figsize=(13, 5))
```

```

ax.bar(x - width, acc_baseline, width, label='Baseline', color='#9467bd', edgecolor='black')
ax.bar(x, acc_unreg, width, label='Unregularized', color='#d62728', edgecolor='black')
ax.bar(x + width, acc_reg, width, label='Regularized', color='#2ca02c', edgecolor='black')
ax.set_ylabel('Accuracy (%)', fontsize=12)
ax.set_xlabel('Digit Class', fontsize=12)
ax.set_title('Per-Class Accuracy Comparison - All Three Models', fontsize=14, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels([str(i) for i in range(10)])
ax.set_ylim(92, 100.2)
ax.legend(fontsize=10)
ax.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

```



Detailed Classification Reports

```
print("Baseline Classification Report:")
print(classification_report(labels_baseline, preds_baseline, target_names=[f"Digit {i}" for i in
range(10)]))

print("Unregularized Classification Report:")
print(classification_report(labels_unreg, preds_unreg, target_names=[f"Digit {i}" for i in
range(10)]))

print("Regularized Classification Report:")
print(classification_report(labels_reg, preds_reg, target_names=[f"Digit {i}" for i in
range(10)]))
```

Baseline Classification Report:

	precision	recall	f1-score	support
Digit 0	0.97	0.99	0.98	1035
Digit 1	0.98	0.98	0.98	1181
Digit 2	0.97	0.96	0.96	1049
Digit 3	0.96	0.96	0.96	1071
Digit 4	0.97	0.96	0.96	1024
Digit 5	0.97	0.97	0.97	947
Digit 6	0.96	0.98	0.97	1032
Digit 7	0.98	0.97	0.98	1094
Digit 8	0.97	0.95	0.96	1024
Digit 9	0.95	0.95	0.95	1043
accuracy			0.97	10500
macro avg	0.97	0.97	0.97	10500
weighted avg	0.97	0.97	0.97	10500

Unregularized Classification Report:

	precision	recall	f1-score	support
Digit 0	0.99	1.00	0.99	1035
Digit 1	0.99	0.99	0.99	1181

Digit 2	0.99	0.98	0.99	1049
Digit 3	0.99	0.97	0.98	1071
Digit 4	0.99	0.97	0.98	1024
Digit 5	0.97	0.99	0.98	947
Digit 6	0.99	0.98	0.99	1032
Digit 7	0.98	0.99	0.98	1094
Digit 8	0.98	0.98	0.98	1024
Digit 9	0.96	0.99	0.97	1043
accuracy			0.98	10500
macro avg	0.98	0.98	0.98	10500
weighted avg	0.98	0.98	0.98	10500
Regularized Classification Report:				
	precision	recall	f1-score	support
Digit 0	0.99	0.99	0.99	1035
Digit 1	0.99	1.00	0.99	1181
Digit 2	0.99	0.98	0.99	1049
Digit 3	0.99	0.99	0.99	1071
Digit 4	0.99	0.98	0.99	1024
Digit 5	0.98	0.99	0.99	947
Digit 6	0.99	0.99	0.99	1032
Digit 7	0.99	0.99	0.99	1094
Digit 8	0.99	0.99	0.99	1024
Digit 9	0.98	0.99	0.98	1043
accuracy			0.99	10500
macro avg	0.99	0.99	0.99	10500
weighted avg	0.99	0.99	0.99	10500

Misclassified Samples Visualization

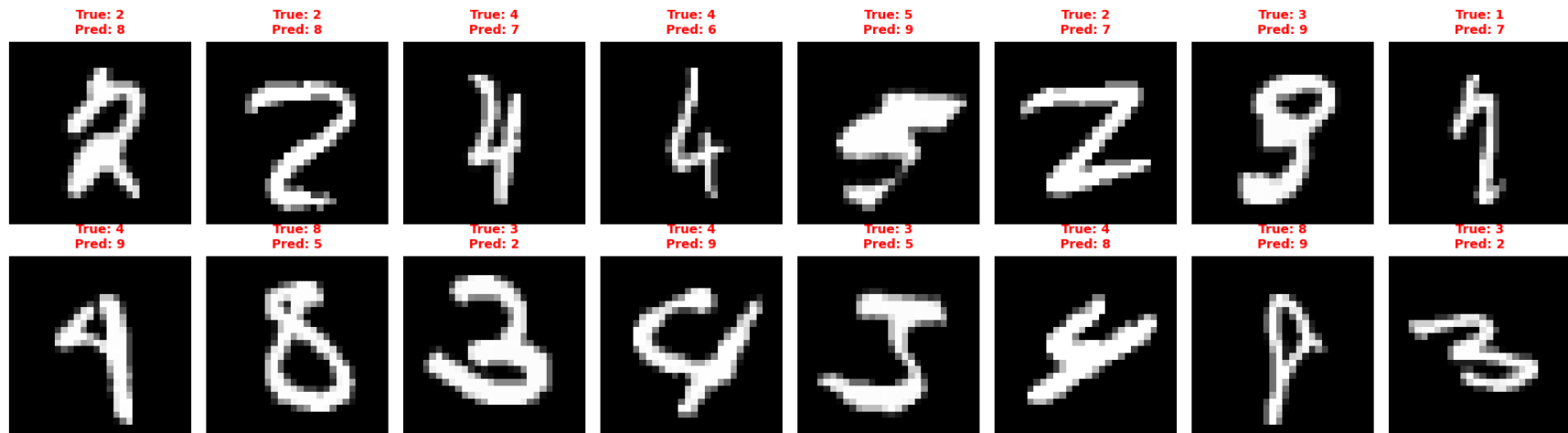
```
errors_unreg = np.where(preds_unreg != labels_unreg)[0]
```

```

if len(errors_unreg) > 0:
    samples_to_show = min(16, len(errors_unreg))
    fig, axes = plt.subplots(2, 8, figsize=(16, 5))
    for i in range(samples_to_show):
        ax = axes[i // 8, i % 8]
        idx = errors_unreg[i]
        image, _ = test_set[idx]
        ax.imshow((image.squeeze() * 0.3081 + 0.1307).numpy(), cmap='gray')
        ax.set_title(f"True: {labels_unreg[idx]}\nPred: {preds_unreg[idx]}", color='red',
        fontsize=9)
        ax.axis('off')
    for i in range(samples_to_show, 16):
        axes[i // 8, i % 8].axis('off')
    plt.suptitle("Sample Misclassified Digits (Unregularized Model)", fontsize=12,
    fontweight='bold')
    plt.tight_layout()
    plt.show()

```

Sample Misclassified Digits (Unregularized Model)



11. Summary and Conclusions

The numbers quoted below are generated directly from the variables computed earlier in this notebook (the code cell right after this one builds the summary programmatically), so they always reflect whatever the most recent run actually produced -- nothing here is hardcoded.

```
from IPython.display import display, Markdown

gap_baseline = t_acc_baseline - v_acc_baseline
gap_unreg     = t_acc_final   - v_acc_final
gap_reg       = t_acc_reg_final - v_acc_reg_final

markdown_content = f"""# Summary and Conclusions

### 1. Baseline as Reference Point
The baseline MLP ({param_count_baseline:}, parameters) reaches {t_acc_baseline:.2f}% train / {v_acc_baseline:.2f}% val accuracy after {BASELINE_EPOCHS} epochs, a generalization gap of {gap_baseline:.2f}%. On the held-out test set it scored {test_acc_baseline:.2f}%. This confirms MNIST is not inherently hard to generalize on when model capacity is well matched to task complexity. Any overfitting seen below is a consequence of excess capacity (not the dataset or the split).

### 2. Empirical Overfitting Through Excess Capacity (Part A)
The unregularized MLP ({experimental_param_count:}, parameters, {experimental_param_count // param_count_baseline}x the baseline) trained for {MAX_EPOCHS} epochs on the identical data reaches {t_acc_final:.2f}% train accuracy vs. only {v_acc_final:.2f}% val accuracy (a gap of {gap_unreg:.2f}%, vs. just {gap_baseline:.2f}% for the baseline). Test accuracy was {test_acc_unreg:.2f}%, and the peak train-val gap of {gap_per_epoch[peak_gap_epoch]:.2f}% occurred at epoch {peak_gap_epoch + 1}. Because this MLP has no convolutional inductive bias (no weight sharing, no spatial locality, and no translation equivariance), every pixel is an independent input feature, and the network is free to memorize exact training-set pixel patterns rather than learning general digit structure.

### 3. Regularization Recovers Generalization (Part B)
With the identical architecture (not one layer changed) but with data augmentation, weight decay (1e-3), a ReduceLROnPlateau scheduler, and early stopping added to the training protocol, the
```

model reaches {t_acc_reg_final:.2f}% train / {v_acc_reg_final:.2f}% val accuracy (a gap of {gap_reg:.2f}%, which is dramatically smaller than the {gap_unreg:.2f}% gap from Part A). Test accuracy was {test_acc_reg:.2f}%.

4. Headline Comparison

Model	Train Acc	Val Acc	Test Acc	Gap (Train - Val)
---	----	----	----	----
Baseline	{t_acc_baseline:.2f}%	{v_acc_baseline:.2f}%	{test_acc_baseline:.2f}%	{gap_baseline:.2f}%
Unregularized	{t_acc_final:.2f}%	{v_acc_final:.2f}%	{test_acc_unreg:.2f}%	{gap_unreg:.2f}%
Regularized	{t_acc_reg_final:.2f}%	{v_acc_reg_final:.2f}%	{test_acc_reg:.2f}%	{gap_reg:.2f}%

Conclusion

The same over-parameterized MLP architecture goes from a {gap_unreg:.2f}% overfitting gap (Part A) down to {gap_reg:.2f}% (Part B) purely through changes in the training protocol (data augmentation, weight decay, learning rate scheduling, and early stopping) with the architecture itself completely unchanged. This demonstrates that generalization is not solely a function of parameter count: a high-capacity network can be made to generalize about as well as (or even better than) a properly-sized baseline when the training recipe compensates for the extra capacity."

```
display(Markdown(markdown_content))
```

```
<IPython.core.display.Markdown object>
```